

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ОТЧЕТ

о преддипломной (производственной) практике

наименование вида и типа практики

на (в) ООО «МЦОБ. Онлайн сервис»

наименование предприятия, организации, учреждения

Студента 4 курса, группы ПО-226

курса, группы

Дмитренко Тимура Александровича

фамилия, имя, отчество

Руководитель практики от
предприятия, организации,
учреждения

Оценка _____

должность, звание, степень

фамилия и. о.

подпись, дата

Руководитель практики от
университета

Оценка _____

К.Т.Н. доцент

должность, звание, степень

Чаплыгин А. А.

фамилия и. о.

подпись, дата

Члены комиссии

подпись, дата

фамилия и. о.

подпись, дата

фамилия и. о.

подпись, дата

фамилия и. о.

Курск 2026 г.

СОДЕРЖАНИЕ

1	Анализ предметной области	6
1.1	Особенности предметной области и назначение разработки	6
1.2	Современные архитектурные подходы	6
1.3	Обзор графических API для разработки игровых движков	7
1.4	Технологический стек его обоснование	10
1.5	Обзор существующих решений	11
1.5.1	Анализ возможностей Unity	11
1.5.2	Анализ возможностей Unreal Engine	12
1.5.3	Анализ возможностей Godot	13
2	Техническое задание	15
2.1	Основание для разработки	15
2.2	Цель и назначение разработки	15
2.3	Библиотека движка	16
2.3.1	Требования к функционалу движка	16
2.3.2	Описание API движка	18
2.3.2.1	Класс CameraManager	18
2.3.2.2	Класс GameObjectManager	19
2.3.2.3	Класс LightManager	21
2.3.2.4	Класс SceneManager	25
2.3.2.5	Класс GameApplication	27
2.3.2.6	Класс BaseScene	28
2.3.2.7	Класс GLBImporter	30
2.3.2.8	Класс STLImporter	30
2.3.3	Пример использования движка	31
2.4	Визуальный редактор	36
2.4.1	Требования к интерфейсу визуального редактора	36
2.4.2	Моделирование вариантов использования	38
2.4.2.1	Вариант использования «Изменение границ камеры»	40

2.4.2.2	Вариант использования «Изменение скорости движения камеры»	40
2.4.2.3	Вариант использования «Изменение чувствительности камеры»	41
2.4.2.4	Вариант использования «Загрузка модели объекта»	42
2.4.2.5	Вариант использования «Просмотр объектов сцены»	42
2.4.2.6	Вариант использования «Удаление объекта»	42
2.4.2.7	Вариант использования «Задание видимости объекта»	43
2.4.2.8	Вариант использования «Настройка свойств объекта»	43
2.4.2.9	Вариант использования «Изменение тэга объекта»	44
2.5	Требования к надежности	44
2.6	Требования к программной и аппаратной совместимости	46
2.7	Операционная система	46
2.8	Аппаратные требования	46
2.9	Программное обеспечение	47
2.10	Требования к графическому API	47
2.11	Требования к производительности	47
2.12	Требования к безопасности	48
2.13	Требования к размещению и сопровождению	48
2.14	Требования к оформлению документации	48
3	Технический проект	49
3.1	Обоснование выбора технологий проектирования	49
3.1.1	Язык программирования C#	49
3.1.2	Графический API OpenGL	51
3.1.3	Windows Forms для реализации редактора сцен	53
3.1.4	Библиотека SixLabors.ImageSharp	55
3.2	Общая структура движка	56
3.3	Взаимодействие основных модулей	60
3.4	Графическая подсистема	62
3.4.1	Управление буферами вершин и индексов	62
3.4.2	Компиляция и загрузка шейдерных программ	62

3.5 Система трансформации	64
3.5.1 Матрицы преобразований	64
3.5.2 Позиция, поворот и масштаб	66
3.6 Система камер	67
3.6.1 Типы проекций (перспективная и ортографическая)	67
3.6.2 Управление положением и направлением обзора	67
3.6.3 Матрица вида и матрица проекции	67
3.7 Система освещения	69
3.7.1 Типы источников света (точечные, направленные, окружающие)	69
3.7.2 Модель освещения	71
3.7.3 Работа с SSBO	72
3.8 Управление сценами	75
3.8.1 Загрузка и выгрузка сцен	75
3.8.2 Сохранение сцены в файл и восстановление	76
3.9 Подсистема обработки ввода	79
3.9.1 Работа хуков	79
3.9.2 Рейкастинг	81
3.10 Система импорта трёхмерных моделей	82
3.10.1 Поддерживаемые форматы (STL, GLB)	82
3.10.2 Парсинг данных из STL-файлов	84
3.10.3 Парсинг данных и иерархии узлов из GLB-файлов	86
3.10.4 Выводы по разделу	91
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	92

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

EBO – element buffer object (буфер индексов вершин).

VAO – vertex array object (объект буфера вершин).

VBO – vertex buffer object (буфер вершин).

GUI – Graphical User Interface (графический интерфейс пользователя) .

GPU – Graphics Processing Unit (графическое процессорное устройство).

FPS – Frames Per Second (кадров в секунду).

SSBO – Shader Storage Buffer Object (буфер хранения данных шейдера).

3D – Трёхмерный.

FPS – Frames per second (Количество кадров в секунду).

JSON – JavaScript Object Notation.

FOV – Field of view (поле зрения).

1. Анализ предметной области

1.1. Особенности предметной области и назначение разработки

Предметной областью данной работы является разработка игровых движков — программных систем, обеспечивающих создание и функционирование видеоигр и интерактивных приложений в реальном времени. Ключевой особенностью предметной области выступает высокая сложность системной интеграции, поскольку игровой движок должен объединять разнородные подсистемы, такие как рендеринг, физика, звук, обработка ввода и управление ресурсами, каждая из которых предъявляет собственные требования к производительности и синхронизации. Другой важнейшей особенностью являются жёсткие требования к работе в реальном времени: современные игры должны обеспечивать частоту смены кадров не менее 60 кадров в секунду, что отводит на обработку одного кадра не более 16,6 миллисекунды, включая все вычисления физики, логики и графики.

1.2. Современные архитектурные подходы

В области разработки игровых движков за последние годы сформировалось несколько архитектурных подходов, каждый из которых имеет свои сильные и слабые стороны. Наиболее традиционным является объектно-ориентированный подход, при котором игровые сущности представляют собой объекты классов, организованные в иерархию наследования. В такой модели базовый класс игрового объекта содержит методы обновления и отрисовки, а конкретные типы объектов (игрок, враг, снаряд) наследуют и переопределяют эти методы. Данный подход интуитивно понятен и широко распространён, однако он приводит к проблеме глубоких иерархий, когда изменение поведения на одном уровне наследования может непредсказуемо повлиять на все дочерние классы. Кроме того, объектно-ориентированный подход затрудняет гибкое комбинирование свойств объекта, что в игровой разработке требуется постоянно, например, когда объект должен одновременно обладать способностью двигаться, атаковать и издавать звук.

В ответ на ограничения объектно-ориентированного подхода получила распространение компонентная архитектура. В этой модели игровой объект представляет собой контейнер, в который можно добавлять отдельные компоненты, отвечающие за конкретные аспекты поведения. Компонент перемещения отвечает за позицию и скорость, компонент рендеринга определяет визуальное представление объекта, компонент здоровья управляет очками прочности, а компонент искусственного интеллекта задаёт логику поведения. Такой подход позволяет гибко конфигурировать объекты без создания сложных иерархий наследования, просто добавляя или удаляя компоненты. Именно компонентную архитектуру используют такие популярные движки, как Unity и Unreal Engine, что подтверждает её эффективность для широкого круга игровых проектов.

1.3. Обзор графических API для разработки игровых движков

Для разработки игровых движков, особенно трёхмерной графики, критически важным является выбор графического прикладного программного интерфейса, поскольку именно через этот интерфейс движок взаимодействует с видеокартой и графическим драйвером для отрисовки изображения. Графический API предоставляет разработчику набор функций для работы с буферами вершин, текстурами, шейдерами и другими ресурсами графического конвейера. На сегодняшний день на рынке существуют четыре основных графических API: OpenGL, Vulkan, DirectX 11, DirectX 12 и Metal, каждый из которых имеет свою историю, область применения и архитектурные особенности.

OpenGL является одним из старейших и наиболее распространённых графических API, разработка которого началась в 1992 году компанией Silicon Graphics и впоследствии перешла под контроль некоммерческого консорциума Khronos Group. Основным преимуществом OpenGL является его кроссплатформенность — он поддерживается на операционных системах Windows, Linux, macOS в устаревшем виде, а также на Android и iOS. OpenGL предоставляет разработчику достаточно высокий уровень абстрак-

ции, беря на себя управление многими низкоуровневыми аспектами работы графического процессора, такими как управление памятью буферов и синхронизация команд. Это делает OpenGL относительно простым в изучении и использовании по сравнению с более современными API, а порог входа для создания простого рендеринга составляет около двухсот-трёхсот строк кода. Однако эта простота является обратной стороной ограниченного контроля над GPU: разработчик не может явно управлять тем, как драйвер планирует выполнение команд, что в определённых сценариях приводит к неоптимальной производительности, особенно при большом количестве отдельных вызовов отрисовки. Кроме того, OpenGL долгое время страдал от проблемы глобального состояния, когда вызов любой функции мог зависеть от ранее установленных параметров, что усложняло многопоточную работу.

DirectX 11 и DirectX 12 являются графическими API, разработанными корпорацией Microsoft исключительно для операционной системы Windows и игровой консоли Xbox. DirectX 11, выпущенный в 2009 году, по своей концепции близок к OpenGL — он также предоставляет достаточно высокий уровень абстракции и относительно прост в использовании. DirectX 11 широко применяется в коммерческих игровых проектах, особенно тех, которые ориентированы исключительно на платформу Windows, и его производительность на этой платформе зачастую превосходит OpenGL благодаря более плотной интеграции с драйверами Windows. DirectX 12, выпущенный в 2015 году, представляет собой кардинально переработанный API низкого уровня, который предоставляет разработчику гораздо более прямой контроль над графическим процессором. В DirectX 12 разработчик самостоятельно управляет выделением памяти для буферов, синхронизацией между командными очередями, а также распределением работы между центральным и графическим процессорами. Это позволяет достичь значительно более высокой производительности в сложных сценах с тысячами объектов, но цена такого контроля — лавинообразный рост сложности разработки. Для создания минимально работающего приложения на DirectX 12 требуется более тысячи строк ко-

да только для инициализации, причём большая часть этого кода посвящена управлению ресурсами и синхронизацией, а не собственно отрисовке.

Vulkan, также разрабатываемый консорциумом Khronos Group, был выпущен в 2016 году как духовный наследник OpenGL, но с архитектурой, вдохновлённой DirectX 12 и улучшениями, призванными сделать его более кроссплатформенным и эффективным. Как и DirectX 12, Vulkan является низкоуровневым API, предоставляющим разработчику явный контроль над графическим конвейером и памятью GPU. Vulkan поддерживается на Windows, Linux, Android и, через сторонние реализации, на macOS. Ключевая идея Vulkan заключается в том, что многие проверки и операции, которые в OpenGL выполнялись драйвером во время работы программы, в Vulkan переносятся на этап инициализации, что снижает накладные расходы во время рендеринга. Кроме того, Vulkan с самого начала проектировался для многопоточности: разные потоки могут параллельно строить командные буферы, которые затем отправляются на выполнение. Основным недостатком Vulkan является его чрезвычайная сложность — объём шаблонного кода для инициализации может достигать нескольких тысяч строк, а любая ошибка в управлении синхронизацией приводит к трудноотлаживаемым проблемам, вплоть до краша драйвера.

Metal является проприетарным графическим API, разработанным компанией Apple и представленным в 2014 году. Metal поддерживается только на операционных системах Apple: macOS, iOS, tvOS и iPadOS. По своей концепции Metal находится между высокоуровневыми API и низкоуровневыми, предоставляя достаточно прямой контроль над GPU, но с меньшим объёмом шаблонного кода по сравнению с Vulkan или DirectX 12. Для разработчиков, ориентированных исключительно на экосистему Apple, Metal является оптимальным выбором, поскольку он тесно интегрирован с графическими драйверами Apple и позволяет добиться наилучшей производительности на этой платформе. Однако для кроссплатформенных проектов Metal неприменим, поскольку он не работает за пределами устройств Apple.

Сравнивая перечисленные API между собой, можно выделить несколько ключевых критериев выбора. По кроссплатформенности лидируют OpenGL и Vulkan, которые поддерживаются на большинстве настольных и мобильных платформ, тогда как DirectX привязан к Windows и Xbox, а Metal — к экосистеме Apple. По сложности освоения и объёму необходимого кода OpenGL является самым доступным, за ним следуют DirectX 11 и Metal, и значительно более сложными являются DirectX 12 и Vulkan. По уровню контроля над GPU и потенциальной производительности OpenGL уступает остальным, а Vulkan и DirectX 12 предоставляют максимальный контроль. По зрелости и распространённости в игровой индустрии OpenGL и DirectX 11 остаются наиболее укоренёнными, хотя новые проекты всё чаще выбирают Vulkan или DirectX 12 для достижения максимальной производительности.

1.4. Технологический стек его обоснование

Для реализации игрового движка в рамках данной дипломной работы был выбран следующий технологический стек. В качестве основного языка программирования используется C#. Этот язык был выбран по нескольким причинам. Во-первых, C# предоставляет управляемую среду выполнения с автоматической сборкой мусора, что существенно снижает риск ошибок, связанных с утечками памяти и висячими указателями, по сравнению с языком C++. Во-вторых, C# обладает богатой стандартной библиотекой, включая удобные коллекции, LINQ для работы с данными, механизмы рефлексии и асинхронного программирования, что ускоряет разработку по сравнению с C++. В-третьих, C# является кроссплатформенным языком благодаря экосистеме .NET, что позволяет при необходимости собирать движок под различные операционные системы. В-четвёртых, выбор C# имеет и образовательную ценность, поскольку большинство коммерческих игровых движков, включая Unreal Engine и Godot до версии 4.0, традиционно пишутся на C++, и разработка движка на C# позволяет провести сравнительный анализ двух

подходов и выявить преимущества и недостатки управляемой среды в контексте графики реального времени.

В качестве графического API выбран OpenGL версии 4.6. Причинами этого выбора являются кроссплатформенность OpenGL, поддерживающего Windows, Linux и macOS, относительно низкий порог входа по сравнению с Vulkan или DirectX 12, а также достаточный уровень контроля над графическим конвейером для реализации всех необходимых функций игрового движка. OpenGL 4.6 включает современные возможности, такие как прямой доступ к состояниям, шейдерные хранимые буферы и вычислительные шейдеры, что позволяет использовать актуальные техники рендеринга. Отказ от Vulkan и DirectX 12 обусловлен их чрезвычайной сложностью, которая привела бы к неоправданному росту объёма шаблонного кода и смещению фокуса с архитектуры игрового движка на низкоуровневое управление ресурсами GPU.

1.5. Обзор существующих решений

1.5.1. Анализ возможностей Unity

Unity является одним из наиболее распространённых игровых движков в мире, особенно в сегменте мобильной разработки, где, по статистике, около семи из каждых десяти мобильных игр созданы с использованием именно этого инструмента. Такая популярность обусловлена выверенным балансом между функциональностью, простотой освоения и кроссплатформенностью. В рамках анализа предметной области целесообразно рассмотреть ключевые технологические возможности Unity, которые делают его референтным решением для сравнения с разрабатываемым движком, а также выделить его ограничения, требующие понимания при выборе архитектурных решений.

Главным преимуществом Unity является его доступность для широкого круга разработчиков, включая независимых студий и одиночных энтузиастов. Движок использует язык C# в качестве основного языка программирования, который значительно проще в освоении по сравнению с C++, приме-

няемым в Unreal Engine, и при этом предоставляет достаточно высокую производительность. Среда разработки Unity отличается интуитивно понятным интерфейсом, а наличие огромного магазина активов с готовыми моделями, скриптами и инструментами позволяет существенно сократить время прототипирования. Эта экосистема делает Unity идеальным выбором для проектов, где важна скорость разработки и быстрая итеративность.

1.5.2. Анализ возможностей Unreal Engine

Unreal Engine, разрабатываемый компанией Epic Games, является одним из самых мощных и функционально насыщенных игровых движков в мире, занимающим лидирующие позиции в сегменте AAA-проектов, то есть игр с крупнейшими бюджетами и наиболее высокими требованиями к графике. В отличие от Unity, который часто выбирают за простоту и доступность, Unreal Engine позиционируется как инструмент для создания визуально впечатляющих проектов, где качество графики является первостепенной задачей. Анализ возможностей Unreal Engine важен для понимания того, какие архитектурные решения и технологии признаны индустрией как передовые, и в какой степени они могут быть адаптированы или осмыслены в рамках разработки собственного учебного движка.

Ключевым отличием Unreal Engine от большинства других движков является то, что его исходный код полностью открыт и доступен для изучения и модификации после подписания стандартного лицензионного соглашения. Это предоставляет разработчикам беспрецедентный уровень контроля: при необходимости можно изменить поведение любой подсистемы, от физики до рендеринга, и пересобрать движок под конкретные нужды проекта. Именно благодаря этой открытости Unreal Engine стал не только инструментом для создания игр, но и мощной образовательной платформой, на примере которой можно изучать устройство промышленного игрового движка. Для данной дипломной работы этот аспект важен, поскольку принципы, реализованные в Unreal Engine, во многом определяют ожидания от архитектуры игровых движков в целом.

В качестве основного языка программирования Unreal Engine использует C++, что является традиционным выбором для высокопроизводительных систем. Однако разработчики Epic Games внедрили мощную систему рефлексии, работающую через специальные макросы и генератор кода Unreal Header Tool, которая добавляет в C++ такие возможности, как сборка мусора, метаинформация о типах, сериализация и механизм событий. Благодаря этому разработчик получает производительность нативного кода с некоторыми удобствами, характерными для управляемых языков. Кроме того, Unreal Engine включает визуальный язык программирования Blueprints, позволяющий создавать игровую логику без написания кода, соединяя узлы, представляющие функции и переменные. Blueprints особенно полезны для дизайнеров уровней и художников, позволяя им быстро прототипировать поведение без участия программистов, а также служат отличным инструментом для визуализации потоков данных и логики.

1.5.3. Анализ возможностей Godot

Godot Engine представляет собой свободно распространяемый игровой движок с открытым исходным кодом, который за последние годы приобрёл значительную популярность, особенно среди независимых разработчиков и энтузиастов. В отличие от Unity и Unreal Engine, которые принадлежат коммерческим компаниям, Godot развивается силами сообщества и некоммерческого фонда Godot, что определяет его философию, лицензионную политику и архитектурные особенности. Анализ возможностей Godot важен для данной работы, поскольку этот движок демонстрирует альтернативный подход к разработке игровых движков, основанный на открытости, модульности и минимализме, и во многом близок по духу к учебному проекту, создаваемому в рамках диплома.

Ключевым преимуществом Godot является его лицензия MIT, которая позволяет использовать, модифицировать и распространять движок без каких-либо отчислений и без необходимости открывать собственный код производных проектов. Это делает Godot идеальным выбором для тех, кто

хочет полностью контролировать свой технологический стек, не опасаясь изменений лицензионной политики, как это произошло с Unity в 2023 году. Для образовательных целей открытость Godot имеет особую ценность, поскольку студенты могут изучать исходный код движка, понимать внутреннее устройство его подсистем и даже вносить в него изменения. Этот аспект роднит Godot с целями данной дипломной работы, где самостоятельная разработка движка также направлена на глубокое понимание его внутренних механизмов.

Архитектурно Godot построен вокруг собственной системы узлов и сцен, которая существенно отличается от компонентной модели Unity или объектной модели Unreal Engine. В Godot всё является узлом, причём каждый узел выполняет конкретную функцию: узел `KinematicBody` отвечает за физическое тело, узел `Sprite` за отображение двухмерной графики, узел `AudioStreamPlayer` за воспроизведение звука. Сцена в Godot представляет собой дерево узлов, которое может быть сохранено в файл и затем использовано как подсистема внутри другой сцены. Это порождает иерархический подход к конструированию игровых объектов, когда сложное поведение собирается из простых узлов, вложенных друг в друга. Такой подход интуитивно понятен и позволяет быстро прототипировать структуру уровней, однако при глубокой вложенности может приводить к сложностям с отслеживанием связей между узлами.

2. Техническое задание

2.1. Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра, посвящённую созданию игрового движка, предназначенного для разработки трёхмерных интерактивных приложений.

Разрабатываемая система создаётся в рамках преддипломной практики как программное средство, обеспечивающее управление графическим рендерингом через OpenGL, обработку пользовательского ввода, загрузку и управление игровыми ресурсами, создание камер и источников освещения для того чтобы создавать игровые приложения без использования сторонних коммерческих движков.

2.2. Цель и назначение разработки

Основной целью работы является разработка и внедрение игрового движка для создания двухмерных и трёхмерных интерактивных приложений в реальном времени.

Назначение разрабатываемой системы заключается в объединении в рамках единой программной платформы управления сценами, обработки иерархии игровых объектов, настройки камер и источников света, а также импорта трёхмерных моделей в форматах STL и GLB с возможностью их визуализации и трансформации в сцене.

Задачами разработки являются:

1. Реализовать графическую подсистему на базе OpenGL, включающую создание окна и контекста рендеринга, загрузку вершинных и индексных буферов, компиляцию шейдерных программ, а также отрисовку трёхмерных объектов.

2. Реализовать систему управления игровыми объектами и компонентами, позволяющую динамически создавать, уничтожать и модифицировать объекты на сцене, а также организовывать их иерархическую структуру.

3. Реализовать подсистему обработки пользовательского ввода с клавиатуры и мыши, обеспечивающую опрос состояния устройств и генерацию событий нажатия и отпускания клавиш.

4. Разработать компонент камеры, поддерживающий ортографическую и перспективную проекции, а также интерактивное управление положением и направлением обзора.

5. Создать демонстрационное приложение, наглядно показывающее все реализованные возможности движка, включая сцену с несколькими объектами и источниками света.

6. Подготовить документацию, описывающую архитектуру движка, интерфейсы программных модулей и инструкцию по созданию демо-сцены на основе разработанного движка.

2.3. Библиотека движка

2.3.1. Требования к функционалу движка

Программный продукт должен обеспечивать выполнение следующих функций:

1. Система должна поддерживать бинарный формат stl и должна прекращать импорт модели, если она представлена ASCII форматом.

2. Импортёр stl моделей должен поддерживать их 5-битные цвета. В случае, когда бит цвета положительный, он должен создавать на их основе массив цветов вершин.

3. Система должна ограничивать максимальный допустимый размер импортируемых моделей. При загрузке слишком большой модели она должна выводить сообщение об ошибке.

4. Система должна автоматически определять узел модели среди других узлов glb файла. При отсутствии узла в списке скелета, узлом модели должен считаться узел с мешем.

5. Импортёр glb-файлов должен поддерживать бинарные данные, закодированные в `uri` в формате строки `base64`. В случае обнаружения таких свойств он должен парсить их в бинарные данные.

6. Импортёр glb-файла должен поддерживать разреженные аксессоры.

7. Импортёр glb-файла должен правильно обрабатывать модели, содержащие более одной сцены. В случае наличия нескольких сцен должен возвращаться из массив.

8. Шейдер игрового объекта должен строиться после добавления этого объекта в сцену. До этого момента объект должен быть без шейдера.

9. У пользователя должна быть возможность менять настройки рендера, глобальные настройки и модель освещения как для отдельных объектов, так и на глобальном уровне. Настройки можно будет менять во время выполнения программы.

10. При добавлении новой камеры, менеджер должен проверять наличие этой камеры в своем кэше. При добавлении той же камеры, в консоль должно выводиться сообщение об ошибке.

11. Шейдеры объекта должны храниться в пуле. Если пользователь создает объект с таким же по структуре материалом, ему должен присваиваться уже существующий шейдер.

12. Шейдеры должны быть индивидуальными под каждый материал. Все различия логики обработки шейдера должны определяться на этапе компиляции, а не во время его работы.

13. У сцены и объектов должны быть хуки для взаимодействия с игрой.

14. Все пользовательские данные должны валидироваться системой. Должны проверяться данные настроек камеры, свойств объектов и света и всех остальных сущностей движка.

2.3.2. Описание API движка

2.3.2.1. Класс CameraManager

Данный класс необходим для управления камерами сцены. В таблице 2.1 представлены методы класса CameraManager.

Таблица 2.1 – Методы класса CameraManager

Имя	Описание
AddCamera(Camera cam)	Добавляет камеру в сцену. cam - камера которую нужно добавить. Возврат - bool, результат добавления.
RemoveCamera(Predicate <Camera> condition)	Удаляет камеру из сцены. condition - условие удаления. Возврат - int, количество удаленных камер.
RemoveCameraAt(int index)	Удаляет камеру из сцены. index - индекс удаляемой камеры. Возврат - bool, результат удаления.
FindCameras(Predicate <Camera> condition)	Получает камеры из сцены. condition - условие поиска. Возврат - IEnumerable<Camera>, коллекция камер.
GetCameraAt(int index)	Получает камеру из сцены. index - индекс получаемой камеры. Возврат - Camera?, получаемая камера.
Clear()	Удаляет все камеры из сцены.

В таблице 2.2 представлены свойства класса CameraManager.

Таблица 2.2 – Свойства класса CameraManager

Имя	Описание
Cameras	Коллекция камер сцены.
CameraCount	Количество камер в сцене.

В таблице 2.3 представлены события класса CameraManager.

Таблица 2.3 – События класса CameraManager

Имя	Описание
CameraAdded	Вызывается при добавлении новой камеры.
CameraRemoved	Вызывается при удалении камеры.
OnClear	Вызывается при срабатывании метода Clear().

2.3.2.2. Класс GameObjectManager

Данный класс нужен для взаимодействия с игровыми объектами сцены. В таблице 2.4 представлены методы класса GameObjectManager.

Таблица 2.4 – Методы класса GameObjectManager

Имя	Описание
RemoveGameObjects (Predicate<DrawableObject> condition)	Удаляет игровые объекты из сцены. condition - условие удаления. Возврат - int, количество удаленных объектов.
AddGameObject(DrawableObject obj)	Добавляет игровой объект в сцену. obj - добавляемый объект.
RemoveGameObjectAt(int index)	Удаляет игровой объект из сцены. index - индекс удаляемого объекта. Возврат - bool, результат удаления.

Продолжение таблицы 2.4

Имя	Описание
FindGameObjects(Predicate <DrawableObject> condition)	Получает игровые объекты из сцены. condition - условие поиска. Возврат - IEnumerable<DrawableObject>, коллекция объектов.
GetGameObjectAt(int index)	Получает игровой объект из сцены. index - индекс получаемого объекта. Возврат - DrawableObject?, полученный объект.
Clear()	Удаляет все игровые объекты из сцены.
PickObject(float mouseX, float mouseY, int screenWidth, int screenHeight, Camera cam)	Проверяет, попал ли курсор на объект. mouseX - x-координата мыши, mouseY - y-координата мыши, screenWidth - ширина экрана, screenHeight - высота экрана, cam - камера, для которой выпускается луч. Возврат - DrawableObject?, объект, на который попал курсор.

В таблице 2.5 представлены свойства класса GameObjectManager.

Таблица 2.5 – Свойства класса GameObjectManager

Имя	Описание
GameObjects	Коллекция игровых объектов сцены.
Count	Количество игровых объектов в сцене.

В таблице 2.6 представлены события класса GameObjectManager.

Таблица 2.6 – События класса GameObjectManager

Имя	Описание
GameObjectAdded	Вызывается при добавлении нового объекта.

Продолжение таблицы 2.6

Имя	Описание
GameObjectRemoved	Вызывается при удалении объекта из сцены.
OnClear	Вызывается при срабатывании метода Clear().

2.3.2.3. Класс LightManager

Данный класс нужен для управления источниками света сцены. В таблице 2.7 представлены методы класса LightManager.

Таблица 2.7 – Методы класса LightManager

Имя	Описание
UpdateDirectionalLightAt (DirectionalLight light, int index)	Обновляет направленный источник света. light - обновляемый источник, index - индекс источника в массиве сцены.
UpdatePointLightAt (PointLight light, int index)	Обновляет точечный источник света. light - обновляемый источник, index - индекс источника в массиве сцены.
UpdateSpotlightAt(Spotlight light, int index)	Обновляет прожекторный источник света. light - обновляемый источник, index - индекс источника света в массиве сцены.
UpdateDirectionalLights (DirectionalLight light, Predicate<DirectionalLight> condition)	Обновляет направленные источники света. light - источник света, который будет заменять целевые источники, condition - условие поиска источников.
UpdatePointLights(PointLight light, Predicate<PointLight> condition)	Обновляет точечные источники света. light - источник света, который будет заменять целевые источники, condition - условие поиска источников.

Продолжение таблицы 2.7

Имя	Описание
UpdateSpotlights(Spotlight light, Predicate<Spotlight> condition)	Обновляет прожекторные источники света. light - источник света, который будет заменять целевые источники, condition - условие поиска источников.
AddDirectionalLight (DirectionalLight light)	Добавляет направленный источник света в сцену. light - добавляемый источник.
AddPointLight(PointLight light)	Добавляет точечный источник света в сцену. light - добавляемый источник.
AddSpotlight(Spotlight light)	Добавляет прожекторный источник света в сцену. light - добавляемый источник.
RemoveDirectionalLights (Predicate<DirectionalLight> condition)	Удаляет направленные источники света из сцены. condition - условие удаления. Возврат - int, количество удаленных источников.
RemovePointLights (Predicate<PointLight> condition)	Удаляет точечные источники света из сцены. condition - условие удаления. Возврат - int, количество удаленных источников.
RemoveSpotlights (Predicate<Spotlight> condition)	Удаляет прожекторные источники света из сцены. condition - условие удаления. Возврат - int, количество удаленных источников.
RemoveDirectionalLightAt(int index)	Удаляет направленный источник света из сцены. index - индекс источника света в массиве сцены. Возврат - bool, результат удаления.
RemovePointLightAt(int index)	Удаляет точечный источник света из сцены. index - индекс источника света в массиве сцены. Возврат - bool, результат удаления.

Продолжение таблицы 2.7

Имя	Описание
RemoveSpotlightAt(int index)	Удаляет прожекторный источник света из сцены. index - индекс источника света в массиве сцены. Возврат - bool, результат удаления.
FindDirectionalLights (Predicate<DirectionalLight> condition)	Получает направленные источники света из сцены. condition - условие поиска. Возврат - IEnumerable<DirectionalLight>, коллекция источников.
FindPointLights (Predicate<PointLight> condition)	Получает точечные источники света из сцены. condition - условие поиска. Возврат - IEnumerable<PointLight>, коллекция источников.
FindSpotlights (Predicate<Spotlight> condition)	Получает прожекторные источники света из сцены. condition - условие поиска. Возврат - IEnumerable<Spotlight>, коллекция источников.
GetDirectionalLightAt(int index)	Получает направленный источник света из сцены. index - индекс источника света в массиве сцены. Возврат - DirectionalLight?, полученный источник.
GetPointLightAt(int index)	Получает точечный источник света из сцены. index - индекс источника света в массиве сцены. Возврат - PointLight?, полученный источник.
GetSpotlightAt(int index)	Получает прожекторный источник из сцены. index - индекс источника света в массиве сцены. Возврат - Spotlight?, полученный источник.

Продолжение таблицы 2.7

Имя	Описание
Clear()	Удаляет все источники света из сцены.

В таблице 2.8 представлены свойства класса LightManager.

Таблица 2.8 – Свойства класса LightManager

Имя	Описание
DirectionalLights	Коллекция направленных источников света сцены.
PointLights	Коллекция точечных источников света сцены.
Spotlights	Коллекция прожекторных источников света сцены.

В таблице 2.9 представлены события класса LightManager.

Таблица 2.9 – События класса LightManager

Имя	Описание
DirLightAdded	Вызывается при добавлении направленного источника света.
DirLightRemoved	Вызывается при удалении направленного источника света.
DirLightUpdated	Вызывается при обновлении направленного источника света.
PointLightAdded	Вызывается при добавлении точечного источника света.
PointLightRemoved	Вызывается при удалении точечного источника света.

Продолжение таблицы 2.9

Имя	Описание
PointLightUpdated	Вызывается при обновлении точечного источника света.
SpotlightAdded	Вызывается при добавлении прожекторного источника света.
SpotlightRemoved	Вызывается при удалении прожекторного источника света.
SpotlightUpdated	Вызывается при обновлении прожекторного источника света.
OnClear	Вызывается при срабатывании метода Clear().

2.3.2.4. Класс SceneManager

Данный класс необходим для управления сценами игры. В таблице 2.10 представлены методы класса SceneManager.

Таблица 2.10 – Методы класса SceneManager

Имя	Описание
LoadSceneByName(string name)	Загружает сцену в игру. name - имя сцены. Для загрузки сцены по имени ее нужно зарегистрировать методом RegisterScene(BaseScene scene).
LoadScene(BaseScene scene)	Загружает сцену в игру. scene - целевая сцена, которую нужно загрузить.
UnloadScene()	Выгружает текущую сцену.
RegisterScene(BaseScene scene)	Регистрирует сцену для ее загрузки по имени. scene - сцена, которую нужно зарегистрировать

Продолжение таблицы 2.10

Имя	Описание
PushScene(BaseScene scene)	Помещает сцену во внутренний стек. Нужен для хранения истории сцен. scene - сцена, которую нужно поместить.
PopScene()	Загружает сцену, находящуюся на вершине стека сцен, которые были туда помещены методом PushScene(BaseScene scene).

В таблице 2.11 представлены свойства класса SceneManager.

Таблица 2.11 – Свойства класса SceneManager

Имя	Описание
CurrentSceneName	Имя активной сцены.
CurrentScene	Загруженная сцена.
Application	Ссылка на экземпляр класса GameApplication, привязанный к этому менеджеру сцен.
ScenePaused	Позволяет поставить на паузу сцену.

В таблице 2.12 представлены события класса SceneManager.

Таблица 2.12 – События класса SceneManager

Имя	Описание
SceneChanged	Вызывается при смене сцены.
SceneLoaded	Вызывается при загрузке сцены.
SceneUnloaded	Вызывается при выгрузке сцены.

2.3.2.5. Класс **GameApplication**

Данный класс используется для запуска приложения, получения некоторых параметров запуска и управления приложением. В таблице 2.13 представлены методы класса **GameApplication**.

Таблица 2.13 – Методы класса **GameApplication**

Имя	Описание
UnloadScene()	Прокси метод SceneManager .
SetBackgroundColor(Color color)	Устанавливает цвет заднего плана сцен. color - целевой цвет.
SetVSync(bool enabled)	Включает или выключает вертикальную синхронизацию.
LoadScene(BaseScene scene)	Прокси метод SceneManager .

В таблице 2.14 представлены свойства класса **GameApplication**.

Таблица 2.14 – Свойства класса **GameApplication**

Имя	Описание
WindowAspectRatio	Соотношение сторон экрана запущенного приложения - ширина/высота.
isSceneLoaded	Показывает, загружена ли сцена.
SceneManager	Ссылка на экземпляр SceneManager , принадлежащий текущему экземпляру.
FPS	Показывает FPS запущенной игры.
EditMode	Показывает в каком режиме запущена игра - режим с запущенным визуальным редактором или без него.
ClearBackground	Устанавливает, нужно ли перерисовывать задний план в запущенной игре.

Продолжение таблицы 2.14

Имя	Описание
Settings	Устанавливает, какие настройки материала должны попадать в шейдер.
ShadingModel	Управляет моделью освещения игры.
GlobalSettings	Позволяет управлять некоторыми глобальными настройками отрисовки игровых объектов.

В таблице 2.15 представлены события класса `GameApplication`.

Таблица 2.15 – События класса `GameApplication`

Имя	Описание
<code>ChangedAspectRatio</code>	Вызывается при изменении размеров окна.

В таблице 2.16 представлены конструкторы класса `GameApplication`.

Таблица 2.16 – Конструкторы класса `GameApplication`

Имя	Описание
<code>GameApplication(string title, bool editMode, float tickTime)</code>	<code>title</code> - название окна, <code>editMode</code> - нужен ли режим редактирования сцены, <code>tickTime</code> - частота обновления цикла физики.

2.3.2.6. Класс `BaseScene`

Данный класс является базовым для пользовательских сцен. В таблице 2.17 представлены методы класса `BaseScene`.

Таблица 2.17 – Методы (хуки) класса BaseScene

Имя	Описание
OnStart()	Вызывается при запуске сцены.
Update(float deltaTime)	Вызывается каждый кадр игры. deltaTime - время с последнего вызова.
FixedUpdate(float fixedDeltaTime)	Вызывается фиксированное в секунду количество раз. deltaTime - время с последнего вызова.
OnExit()	Вызывается при выгрузке сцены.
OnPause()	Вызывается, когда сцена ставится на паузу.
OnResume()	Вызывается, когда сцена снимается с паузы.
OnKeyDown (KeyboardKeyEventArgs e)	Вызывается, когда в запущенной игре клавиша опускается вниз. e - информация о клавише.
OnKeyUp (KeyboardKeyEventArgs e)	Вызывается, когда в запущенной игре нажатая клавиша поднимается. e - информация о клавише.
OnMouseDown (MouseButtonEventArgs e)	Вызывается, когда в запущенной игре опускается кнопка мыши. e - информация о кнопке.
OnMouseUp (MouseButtonEventArgs e)	Вызывается, когда в запущенной игре поднимается кнопка мыши. e - информация о кнопке.
OnMouseWheel (MouseWheelEventArgs e)	Вызывается, когда в запущенной игре пользователь крутит колесико мыши. e - информация о событии.
OnMouseMove (MouseMoveEventArgs e)	Вызывается, когда в запущенной игре пользователь двигает мышью. e - информация о событии.

В таблице 2.18 представлены свойства класса BaseScene.

Таблица 2.18 – Свойства класса BaseScene

Имя	Описание
GameObjectManager	Ссылка на экземпляр менеджера объектов.
CameraManager	Ссылка на экземпляр менеджера камер.
LightManager	Ссылка на экземпляр менеджера света.
SceneManager	Ссылка на экземпляр менеджера сцен.

2.3.2.7. Класс GLBImporter

Данный класс нужен для импорта .glb моделей. В таблице 2.19 представлены методы класса GLBImporter.

Таблица 2.19 – Методы класса GLBImporter

Имя	Описание
LoadModel(string path, RenderSettings settings = RenderSettings.All)	Загружает модель в игру. path - путь к файлу. settings - свойства материала, которые нужно импортировать. Возврат - GLBScene[]?, набор glb-сцен файла.

2.3.2.8. Класс STLImporter

Данный класс нужен для импорта .stl моделей. В таблице 2.20 представлены методы класса STLImporter.

Таблица 2.20 – Методы класса STLImporter

Имя	Описание
LoadModel(string path, RenderSettings settings = RenderSettings.All)	Загружает модель в игру. path - путь к файлу. settings - флаг, означающий, нужно ли импортировать цвета полигонов. Возврат - STLModel?, загруженная модель.

2.3.3. Пример использования движка

Для подтверждения работоспособности разработанного игрового движка и наглядной демонстрации его основных возможностей было создано демонстрационное приложение, представляющее собой простую 3D игру. Игровой процесс заключается во взаимодействии пользователя с объектами на сцене: при нажатии левой кнопки мыши выполняется выбор объекта, на который указывает курсор, после чего этот объект удаляется из сцены и мгновенно возрождается в новой случайной позиции в пределах игрового пространства. Такая механика позволяет циклически тестировать работу подсистем управления игровыми объектами и обработки пользовательского ввода в динамически изменяющейся сцене.

Демонстрационное приложение использует следующие возможности разработанного движка. Подсистема обработки ввода обеспечивает получение координат курсора в момент клика и определение выбранного объекта в 3D пространстве. Система управления игровыми объектами отвечает за удаление и создание объектов в реальном времени с корректным освобождением ресурсов OpenGL. Система трансформации позволяет задавать объектам случайные позиции при возрождении. Сцена содержит источник освещения, демонстрируя работу системы света движка.

Класс сцены игры.

```

1 public class ExampleScene : BaseScene
2 {
3     private readonly Random _r = new();
4
5     [AllowNull]
```

```

6  private GLBModel _model;
7
8  [AllowNull]
9  private Camera _cam;
10
11 protected override void OnStart()
12 {
13     base.OnStart();
14
15     _model = GLBImporter.LoadModel(@"C:\Users\it_ge\Desktop\Models\rick.glb")![0].Models[1];
16
17     for (int i = 0; i < 10; i++)
18     {
19         SpawnObject();
20     }
21
22     _cam = new Camera(CameraTypes.Perspective, Vector3.Zero, SceneManager!.Application!.
        WindowAspectRatio)
23     {
24         Far = 500.0f,
25         Near = 10.0f
26     };
27
28     CameraManager.AddCamera(_cam);
29
30     DirectionalLight light = new()
31     {
32         Color = Color.White,
33         Intensity = 1.0f,
34         Yaw = -60.0f,
35         Pitch = -30.0f,
36     };
37
38     LightManager.AddDirectionalLight(light);
39 }
40
41 protected override void OnMouseDown(MouseButtonEventArgs e)
42 {
43     base.OnMouseDown(e);
44
45     if (e.Button == MouseButton.Left)
46     {
47         GameApplication app = SceneManager!.Application!;
48         MouseState state = app.MouseState;
49         DrawableObject? pickedObj = GameObjectManager.PickObject(state.X, state.Y, app.ClientSize.X, app
            .ClientSize.Y, _cam);
50
51         if (pickedObj is not null)
52         {
53             GameObjectManager.RemoveGameObject(pickedObj);
54             SpawnObject();
55         }
56     }
57 }
58
59 private void SpawnObject()
60 {
61     GLBGameObject newObject = new(_model);

```

```

62     newObject.Transform.Position = new Vector3(_r.Next(-30, 30), _r.Next(-20, 20), _r.Next(-100, -10))
        ;
63     newObject.Transform.Rotation = new Vector3(0.0f, 0.0f, 90.0f);
64     newObject.Transform.Scale = new Vector3(5.0f);
65     GameObjectManager.AddGameObject(newObject);
66 }
67 }

```

Главный класс программы.

```

1 internal class Program
2 {
3     [STAThread]
4     public static void Main()
5     {
6         GameApplication app = new("ExampleApp", editMode: false);
7
8         app.LoadScene(new ExampleScene());
9         app.Run();
10    }
11 }

```

```

1     [AllowNull]
2     private GLBModel _model;
3
4     [AllowNull]
5     private Camera _cam;
6
7     [AllowNull]
8     private FreeCameraController _camController;

```

Сначала в коде игры объявляются поля, необходимые для ее работы.

`_model` - это загруженная модель игрового объекта, `_cam` - камера сцены.

```

1 _model = GLBImporter.LoadModel(@"C:\Users\it_ge\Desktop\Models\rick.glb")![0].Models[1];

```

В начале метода `OnStart()` модель загружается в игру с помощью импортера. Метод возвращает набор GLB сцен. Так как в файле всего одна сцена и модель, используются операторы `[0].Models[1]` для ее получения.

```

1 private void SpawnObject()
2 {
3     GLBGameObject newObject = new(_model);
4     newObject.Transform.Position = new Vector3(_r.Next(-30, 30), _r.Next(-20, 20), _r.Next(-100, -10));
5     newObject.Transform.Rotation = new Vector3(0.0f, 0.0f, 90.0f);
6     newObject.Transform.Scale = new Vector3(5.0f);
7     GameObjectManager.AddGameObject(newObject);
8 }

```

Далее в цикле вызывается метод `SpawnObject`, который генерирует изначальные объекты сцены. Объектам задаются свойства `Transform.Position`, `Transform.Rotation` и `Transform.Scale`, которые задают положение объекта, по-

ворот и масштаб. Далее созданный объект помещается в сцену командой `GameObjectManager.AddGameObject(newObject);`.

```
1 _cam = new Camera(CameraTypes.Perspective, Vector3.Zero, SceneManager!. Application!. WindowAspectRatio)
2 {
3     Far = 500.0f,
4     Near = 10.0f
5 };
6
7 CameraManager.AddCamera(_cam);
```

Далее в коде создается и добавляется в сцену камера. Аргумент `CameraTypes.Perspective` задает тип камеры - камера с перспективной проекцией. `Vector3.Zero` - это положение камеры в мировых координатах, а 3 аргумент задает соотношение сторон экрана. `Far` и `Near` задают дальнюю и ближнюю границы камеры соответственно.

Команда `CameraManager.AddCamera(_cam);` добавляет камеру в сцену.

```
1 DirectionalLight light = new()
2 {
3     Color = Color.White,
4     Intensity = 1.0f,
5     Yaw = -60.0f,
6     Pitch = -30.0f,
7 };
8
9 LightManager.AddDirectionalLight(light);
```

В данном фрагменте кода создается и добавляется в сцену направленный источник света. `Color` - цвет света, `Intensity` - его интенсивность, `Yaw` и `Pitch` - углы Эйлера направления света.

Команда `LightManager.AddDirectionalLight(light);` Добавляет источник света в сцену.

```
1 protected override void OnMouseDown(MouseButtonEventArgs e)
2 {
3     base.OnMouseDown(e);
4
5     if (e.Button == MouseButton.Left)
6     {
7         GameApplication app = SceneManager!. Application!;
8         MouseState state = app.MouseState;
9         DrawableObject? pickedObj = GameObjectManager.PickObject(state.X, state.Y, app.ClientSize.X, app.
            ClientSize.Y, _cam);
10
11         if (pickedObj is not null)
12         {
13             GameObjectManager.RemoveGameObject(pickedObj);
14             SpawnObject();
15         }
16     }
```

```
16 }  
17 }
```

Данный метод является хуком, который срабатывает при нажатии на кнопку мыши

```
1 GameApplication app = SceneManager!.Application!;  
2 MouseState state = app.MouseState;
```

Данные команды получают экземпляр класса GameApplication игры и получают из нее состояние мыши.

```
1 DrawableObject? pickedObj = GameObjectManager.PickObject(state.X, state.Y, app.ClientSize.X, app.  
  ClientSize.Y, _cam);
```

Эта команда с помощью ray casting определяет, попал ли объект под курсор мыши при нажатии на левую кнопку и в случае попадания получает этот объект.

```
1 if (pickedObj is not null)  
2 {  
3     GameObjectManager.RemoveGameObject(pickedObj);  
4     SpawnObject();  
5 }
```

В конце этого метода производится проверка, попал ли курсор на объект. И если попал, то объект удаляется из сцены и пересоздается.

```
1 GameApplication app = new("ExampleApp", editMode: false);
```

В главном классе игры, в методе Main эта команда создает экземпляр приложения. В него передаются имя окна и bool-значение, означающее, нужно ли запускать визуальный редактор.

```
1 app.LoadScene(new ExampleScene());  
2 app.Run();
```

Далее в игру загружается сцена и окно запускается. Еще важно отметить, что у метода Main должен быть атрибут [STAThread] для правильной работы редактора.

2.4. Визуальный редактор

2.4.1. Требования к интерфейсу визуального редактора

Интерфейс редактора сцен должен представлять собой несколько окон, наложенных поверх игры с единым стилем оформления, обеспечивающее понятное управление над объектами сцены и их компонентами.

Компоненты интерфейса редактора состоят из:

- главное окно редактора для просмотра объектов сцены и загрузки моделей;
- окно инспектора объектов, позволяющее управлять их состоянием;
- кнопку загрузки моделей из файлов;
- текстовые поля для управления настройками камеры редактора;
- список игровых объектов сцены хранящий их превью;
- кнопку "удалить объект" для удаления объекта из сцены;
- кнопку "назад" для выхода из окна инспектора;
- чекбокс для задания видимости объекта;
- текстовые поля для задания объекту свойств положения, поворота и масштаба
- текстовое поле для задания объекту тэга.

Интерфейс должен отображаться при запуске игры с флагом режима редактирования. В данном режиме должны быть доступны функции импорта трёхмерных моделей, настройки параметров камеры редактора, управления списком игровых объектов и редактирования их компонентов через инспектор. В режиме игры данный интерфейс должен отсутствовать, поэтому в данном случае освещение и камеры должны создаваться из кода сцены.

Особое внимание должно быть уделено удобству работы со списком превью загруженных объектов, с инспектором объекта, содержащим поля для трансляции, поворота и масштабирования, а также с параметрами настройки камеры редактора, включающими чувствительность, скорость перемещения, ближнюю и дальнюю дистанции отсечения. Интерфейс должен быть адаптирован для последовательного выполнения действий по импорту

модели, её размещению в сцене, настройке трансформации и тегированию объекта без лишних переходов между несвязанными окнами.

На рисунке 2.1 представлен внешний вид главного меню редактора.

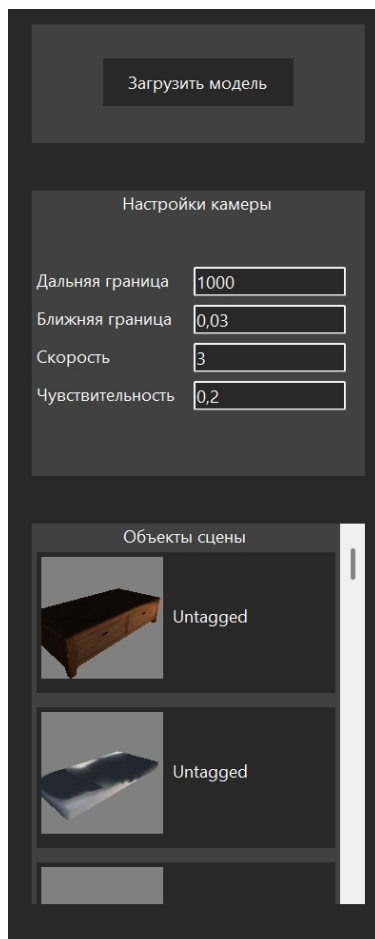


Рисунок 2.1 – Главное меню

Как показано на рисунке, главное меню содержит кнопку для загрузки моделей, настройки камеры редактора, а также список из картинок которые обозначают внешний вид игровых объектов в сцене.

На рисунке 2.2 представлен внешний вид инспектора объектов.

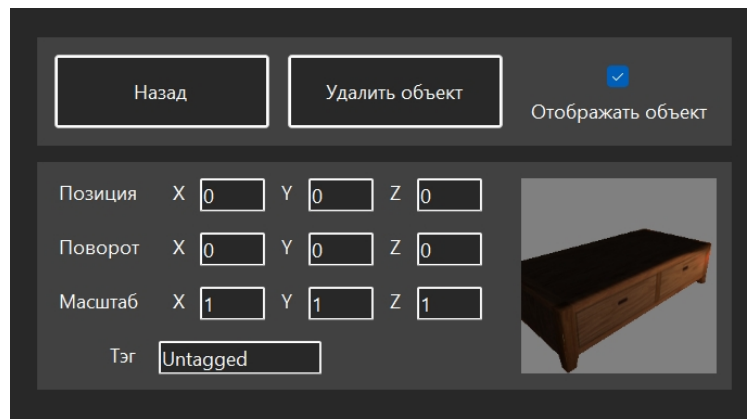


Рисунок 2.2 – Окно инспектора игровых объектов

Окно инспектора игровых объектов включает в себя элементы интерфейса для управления выбранным объектом. Сюда входят поля для ввода transform-свойств, а также другие настройки объекта.

2.4.2. Моделирование вариантов использования

На основании анализа предметной области в программе должны быть реализованы следующие прецеденты:

- изменение границ камеры;
- изменение скорости движения камеры;
- изменение чувствительности камеры;
- загрузка модели объекта;
- просмотр объектов сцены;
- удаление объекта;
- задание видимости объекта;
- настройка свойств объекта;
- изменение тэга объекта.

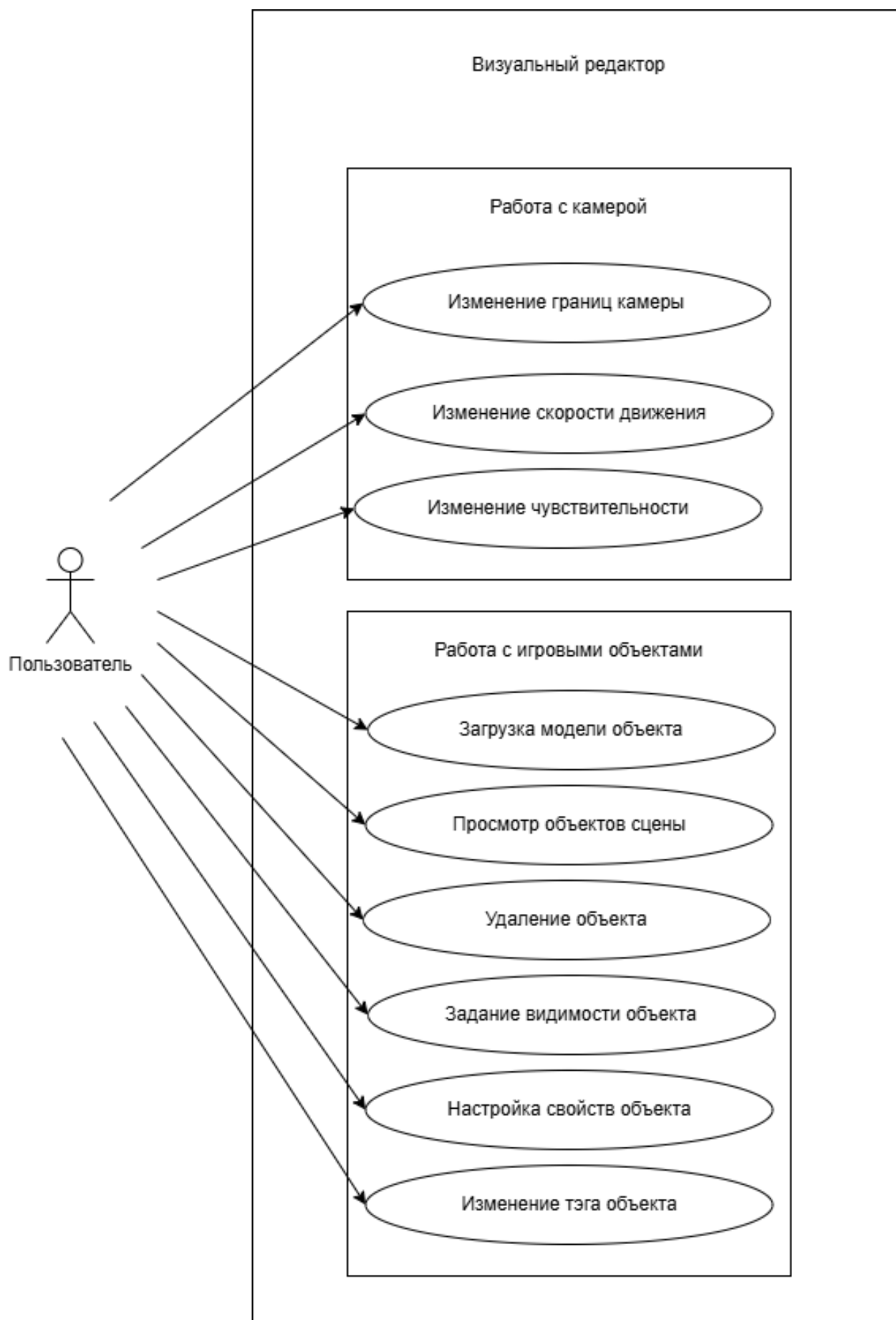


Рисунок 2.3 – Диаграмма прецедентов пользователя

2.4.2.1. Вариант использования «Изменение границ камеры»

Заинтересованные лица и их требования: пользователь желает изменить дальность рендеринга объектов.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: граница камеры успешно изменена.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.
3. Пользователь вводит новое значение в поля для ввода.
4. Система проверяет введенные данные.
5. Система меняет свойство камеры.

Альтернативный сценарий (неверные данные):

1. Пользователь вводит отрицательное значение.
2. Система проверяет значение.
3. Свойство камеры остается предыдущим.

Альтернативный сценарий (пустое поле):

1. Пользователь стирает поле настройки камеры.
2. Система оставляет предыдущее значение настройки.

2.4.2.2. Вариант использования «Изменение скорости движения камеры»

Заинтересованные лица и их требования: пользователь желает изменить скорость перемещения по сцене.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: скорость камеры успешно изменена.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.

3. Пользователь вводит новое значение в поля для ввода.
4. Система проверяет введенные данные.
5. Система меняет скорость движения камеры.

Альтернативный сценарий (неверные данные):

1. Пользователь вводит отрицательное значение.
2. Система проверяет значение
3. Скорость камеры остается предыдущей.

Альтернативный сценарий (пустое поле):

1. Пользователь стирает поле настройки камеры.
2. Система оставляет предыдущее значение настройки.

2.4.2.3. Вариант использования «Изменение чувствительности камеры»

Заинтересованные лица и их требования: пользователь желает изменить чувствительность управления камерой.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: чувствительность камеры успешно изменена.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.
3. Пользователь вводит новое значение в поля для ввода.
4. Система проверяет введенные данные.
5. Система меняет чувствительность камеры.

Альтернативный сценарий (неверные данные):

1. Пользователь вводит отрицательное значение.
2. Система проверяет значение
3. Чувствительность камеры остается предыдущей.

Альтернативный сценарий (пустое поле):

1. Пользователь стирает поле настройки камеры.
2. Система оставляет предыдущее значение настройки.

2.4.2.4. Вариант использования «Загрузка модели объекта»

Заинтересованные лица и их требования: пользователь желает загрузить в сцену игровой объект.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: игровой объект загружен.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.
3. Пользователь нажимает кнопку «Загрузить модель».
4. Пользователь выбирает нужный файл в проводнике.
5. Система загружает модель и на ее основе создает объект.

2.4.2.5. Вариант использования «Просмотр объектов сцены»

Заинтересованные лица и их требования: пользователь желает загрузить в сцену игровой объект.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: пользователю отображаются загруженные объекты.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.
3. Пользователь видит объекты в главном окне.

2.4.2.6. Вариант использования «Удаление объекта»

Заинтересованные лица и их требования: пользователь желает удалить игровой объект из сцены.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: игровой объект удален из сцены.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.
3. Пользователь нажимает на нужный объект в главном окне.
4. Система открывает окно инспектора объекта.
5. Пользователь нажимает на кнопку «Удалить объект».
6. Система удаляет объект из сцены.

2.4.2.7. Вариант использования «Задание видимости объекта»

Заинтересованные лица и их требования: пользователь хочет задать видимость игрового объекта в сцене.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: игровой объект скрыт или показан.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.
3. Пользователь нажимает на нужный объект в главном окне.
4. Система открывает окно инспектора объекта.
5. Пользователь нажимает на чекбокс «Отображать объект».
6. Система задает видимость объекту.

2.4.2.8. Вариант использования «Настройка свойств объекта»

Заинтересованные лица и их требования: пользователь хочет задать видимость игрового объекта в сцене.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: игровой объект изменяет свои свойства.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.

3. Пользователь нажимает на нужный объект в главном окне.
4. Система открывает окно инспектора объекта.
5. Пользователь вводит значения в поля свойств объекта».
6. Система задает объекту новые свойства.

Альтернативный сценарий (пустые поля):

1. Пользователь стирает поля свойств объекта.
2. Система оставляет предыдущее значение свойств.

2.4.2.9. Вариант использования «Изменение тэга объекта»

Заинтересованные лица и их требования: пользователь хочет изменить тэг объекта в сцене.

Предусловие: пользователь запустил приложение в режиме редактирования.

Постусловие: игровой объект получает новый тэг.

Основной успешный сценарий:

1. Пользователь загружает сцену в движок.
2. Пользователь запускает окно с игрой.
3. Пользователь нажимает на нужный объект в главном окне.
4. Система открывает окно инспектора объекта.
5. Пользователь вводит значения в поле «Тэг».
6. Система задает объекту новый тэг.

2.5. Требования к надежности

Система должна обеспечивать надежную работу при различных режимах функционирования, включая редактирование сцены и игровой режим.

При возникновении ошибок в процессе выполнения операций импорта моделей, загрузки сцены и рендеринга система должна выводить диагностическое сообщение в консоль, содержащее информацию о типе ошибки и месте её возникновения. Критические ошибки, приводящие к невозможности дальнейшей работы, должны корректно завершать приложение с сохранением целостности файлов сцены и других пользовательских данных. В случае

некорректного ввода параметров в поля инспектора объекта (нечисловые значения, пустые поля, выход за допустимые пределы) система должна игнорировать некорректные изменения и сохранять предыдущие корректные значения.

На уровне приложения должна быть реализована валидация вводимых пользователем данных. Проверке подлежат числовые значения в полях трансляции, поворота и масштаба объекта (проверка на соответствие числовому формату), параметры камеры редактора (чувствительность, скорость, ближняя и дальняя дистанции должны находиться в допустимых пределах), а также тег объекта (проверка на недопустимые символы и максимальную длину). Это позволяет предотвратить некорректные изменения параметров сцены и возможные ошибки рендеринга.

Система должна обеспечивать устойчивость к ошибочным действиям пользователя, связанным с попыткой импорта неподдерживаемого формата файла, импортом повреждённой модели, вводом некорректных числовых значений в поля трансформации, а также попыткой сохранения сцены при отсутствии изменений. Во всех подобных случаях система должна выводить соответствующее сообщение в консоль и корректно завершать свою работу.

При возникновении ошибок и исключительных ситуаций система должна обеспечивать вывод диагностических сообщений в консоль. Сообщения должны содержать информацию о типе ошибки. При запуске приложения из командной строки или среды разработки все сообщения выводятся в стандартный поток вывода для последующего анализа. Для критических ошибок OpenGL, связанных с потерей контекста или несовместимостью версий, система должна выводить развёрнутое описание проблемы.

Система должна обеспечивать корректное управление как управляемой памятью .NET, так и неуправляемыми ресурсами OpenGL (буферы вершин, текстурные объекты, шейдерные программы). При удалении игрового объекта из сцены соответствующие неуправляемые ресурсы должны немедленно освобождаться через вызов соответствующих функций OpenGL. При завершении приложения все неуправляемые ресурсы должны быть освобождены

во избежание утечек памяти. Система должна минимизировать количество аллокаций в управляемой куче во время основного цикла рендеринга для предотвращения вызовов сборщика мусора, которые могут привести к заметным фризам.

2.6. Требования к программной и аппаратной совместимости

Система должна функционировать как десктопное приложение под управлением операционной системы Windows и предъявлять умеренные требования к аппаратному обеспечению, характерные для современных персональных компьютеров.

2.7. Операционная система

Разрабатываемый игровой движок и редактор сцен могут функционировать под управлением операционной системы Windows (версии 10 и выше). Для корректной работы графической подсистемы требуется наличие установленных драйверов видеокарты с поддержкой OpenGL версии 4.6.

2.8. Аппаратные требования

Для работы разрабатываемого игрового движка и редактора сцен требуется персональный компьютер, обеспечивающий:

- процессор с тактовой частотой не ниже 2,5 ГГц (архитектура x86 или x64);
- оперативную память объёмом не менее 8 ГБ (рекомендуется 16 ГБ для работы со сложными трёхмерными сценами);
- видеокарту с поддержкой OpenGL 4.6 и объёмом видеопамяти не менее 2 ГБ (рекомендуется 4 ГБ и выше);
- наличие свободного дискового пространства не менее 500 МБ для размещения исполняемых файлов движка, библиотек и пользовательских ресурсов (модели, текстуры, сцены);
- устройство ввода: клавиатура и мышь.

2.9. Программное обеспечение

Для работы разработанного игрового движка и редактора сцен не требуется установка дополнительного программного обеспечения или сторонних библиотек. Все необходимые компоненты поставляются вместе с приложением. На целевой машине должна быть установлена среда выполнения .NET 6.0 или выше.

Для конечного пользователя (разработчика игр) для работы с движком и редактором сцен достаточно запустить исполняемый файл приложения. Все зависимости уже включены в дистрибутив и не требуют отдельной установки или настройки.

2.10. Требования к графическому API

Движок предъявляет следующие требования к графической подсистеме компьютера:

- поддержка OpenGL версии не ниже 4.6 (core profile);
- поддержка шейдеров версии 430 и выше;
- объём видеопамяти, достаточный для хранения загруженных текстур и буферов вершин текущей сцены.

2.11. Требования к производительности

Система должна обеспечивать:

- частоту кадров не менее 60 FPS при отрисовке сцены, содержащей до 500 простых трёхмерных объектов (кубы, сферы) с базовыми материалами;
- время загрузки GLB-модели объёмом до 50 МБ не более 2 секунд;
- время переключения между простыми сценами не более 1 секунды;
- потребление оперативной памяти не более 4 ГБ при работе со сценой среднего размера (500 объектов);
- отсутствие заметных фризов, вызванных сборкой мусора, при длительной работе редактора.

2.12. Требования к безопасности

Система должна обеспечивать:

- проверку импортируемых файлов моделей на наличие потенциально опасных данных (неправильные размеры чанков, неправильный формат .stl модели);
- ограничение максимального размера импортируемого файла модели для предотвращения исчерпания ресурсов системы;
- обработку исключительных ситуаций без доступа к системным ресурсам, выходящим за пределы приложения;
- отсутствие необходимости в правах администратора для запуска и работы приложения.

2.13. Требования к размещению и сопровождению

Система должна поставляться в виде набора исполняемых файлов, библиотек и ресурсов, пригодных для запуска на целевом компьютере без выполнения процедуры установки. Для сопровождения системы должны быть доступны исходные файлы проекта на языке C#, файлы ресурсов (модели, текстуры, шейдеры), а также документация по сборке и модификации проекта.

2.14. Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102–77 и ГОСТ 34.601–90. Единая система программной документации.

3. Технический проект

3.1. Обоснование выбора технологий проектирования

3.1.1. Язык программирования C#

В качестве основного языка программирования для реализации игрового движка и редактора сцен выбран язык C#. Данный выбор обусловлен совокупностью факторов, включающих производительность, безопасность памяти, скорость разработки и доступность инструментальной поддержки. C# является объектно-ориентированным языком с управляемой средой выполнения, что обеспечивает автоматическое управление памятью через сборщик мусора. Это позволяет снизить риск ошибок, связанных с утечками памяти и висячими указателями, которые характерны для языков с ручным управлением памятью, таких как C++. В контексте разработки игрового движка, где надёжность и долговременная стабильность работы имеют большое значение, автоматическое управление памятью существенно упрощает разработку и отладку.

Ключевым преимуществом C# является богатая стандартная библиотека и экосистема .NET. Стандартная библиотека предоставляет удобные коллекции, средства работы с файловой системой, потоками и асинхронным программированием. Механизмы рефлексии позволяют реализовать гибкую систему сериализации сцен и компонентов без необходимости написания шаблонного кода. Система LINQ упрощает выполнение операций над коллекциями игровых объектов при реализации игровой логики. Кроме того, среда выполнения .NET включает средства низкоуровневого взаимодействия с нативным кодом через механизм P/Invoke, что необходимо для вызова функций OpenGL и управления окнами через библиотеку GLFW.

Важным аспектом выбора C# является его производительность, достаточная для разработки игровых движков среднего уровня сложности. Благодаря Just-In-Time компиляции в сочетании с оптимизациями на уровне среды выполнения, код на C# может достигать производительности, приближаю-

щейся к нативному коду на C++. При этом современные версии .NET (6.0 и выше) включают аппаратные векторы, интринсики и возможности low-level оптимизаций, позволяющие эффективно работать с массивами вершинных данных и матрицами преобразований. Критически важные для производительности участки, такие как обработка вершинных буферов и расчёт матриц трансформации, могут быть реализованы с использованием типов-структур, что позволяет избежать аллокаций в управляемой куче и снизить нагрузку на сборщик мусора.

Для разработки игрового движка также важна кроссплатформенность языка C#. Хотя в рамках данной работы основной целевой платформой является Windows, код движка спроектирован таким образом, что его можно скомпилировать и запустить под управлением Linux или macOS с минимальными изменениями. Это обеспечивается использованием кроссплатформенных библиотек (GLFW, OpenGL) и независимостью от платформозависимых API Windows. Среда выполнения .NET доступна на всех основных операционных системах благодаря проекту .NET Core, что даёт возможность портирования движка на другие платформы при необходимости.

Наконец, выбор C# обусловлен также удобством разработки графического интерфейса редактора сцен с использованием Windows Forms. Данная технология является зрелой, хорошо документированной и тесно интегрированной с C#, что позволяет быстро создавать функциональные окна, панели инструментов, текстовые поля и списки с минимальными затратами времени. Использование единого языка программирования как для движка, так и для редактора сцен упрощает поддержку и развитие проекта, поскольку разработчику не требуется переключаться между разными языками и инструментами. Таким образом, язык C# обеспечивает оптимальный баланс между производительностью, надёжностью и скоростью разработки для целей дипломного проекта.

3.1.2. Графический API OpenGL

В качестве основного графического интерфейса приложения для реализации рендеринга выбран OpenGL версии 4.6. Данный выбор обусловлен совокупностью факторов, включающих кроссплатформенность, достаточный уровень контроля над графическим конвейером, относительную простоту освоения и наличие обширной документации. OpenGL является открытым стандартом, разрабатываемым консорциумом Khronos Group, и поддерживается на всех современных операционных системах, включая Windows, Linux и macOS, что позволяет в перспективе портировать разрабатываемый движок на другие платформы без замены графического ядра.

Основным преимуществом OpenGL по сравнению с альтернативными API, такими как Vulkan или DirectX 12, является умеренная сложность реализации. Для создания минимального работающего приложения на OpenGL требуется около двухсот-трёхсот строк кода, в то время как для Vulkan аналогичный объём функциональности потребовал бы нескольких тысяч строк только для инициализации. В контексте дипломной работы с ограниченными временными ресурсами это позволяет сосредоточиться на реализации непосредственной функциональности игрового движка, а не на решении низкоуровневых проблем управления синхронизацией и распределения памяти. OpenGL при этом сохраняет достаточный контроль над графическим конвейером для реализации трёхмерной сцены с источниками света, текстурами и трансформациями объектов.

Версия OpenGL 4.6 была выбрана как минимальная целевая версия по следующим причинам. Данная версия включает Direct State Access, позволяющий изменять состояние объектов без привязки к контексту, что упрощает многопоточную работу с ресурсами. Поддерживаются шейдерные хранимые буферы, предоставляющие возможность эффективно передавать большие объёмы данных в шейдеры. Вычислительные шейдеры позволяют выполнять общие вычисления на графическом процессоре без привязки к графическому конвейеру. Кроме того, OpenGL 4.6 является современным стан-

дартом, поддерживаемым большинством видеокарт, выпущенных после 2017 года, включая все дискретные видеокарты NVIDIA серии GeForce 900 и новее, а также AMD Radeon серии R9 300 и новее. Это обеспечивает достаточную совместимость с оборудованием, доступным в учебных аудиториях и у потенциальных пользователей.

Для взаимодействия с OpenGL из управляемого кода C# используется механизм P/Invoke, позволяющий вызывать нативные функции, экспортируемые динамическими библиотеками драйвера видеокарты. Для упрощения работы с функциями OpenGL используется библиотека OpenTK, предоставляющая корректные сигнатуры всех функций OpenGL, а также типы данных, совместимые с GLSL по именованию и структуре. Альтернативой могло бы быть использование библиотеки Silk.NET, однако OpenTK выбрана благодаря её большей распространённости и стабильности. Критически важные вызовы OpenGL, такие как отрисовка буферов и смена шейдерных программ, выполняются часто, и каждый такой вызов является переходом через границу управляемого и неуправляемого кода. Для снижения накладных расходов на межъязыковые вызовы в движке используется пакетирование операций: множество вершинных данных передаётся в GPU одним вызовом, а не серией мелких операций.

Архитектурно работа с OpenGL в движке организована по принципу инкапсуляции. Классы, представляющие графические ресурсы, содержат идентификаторы объектов OpenGL и реализуют интерфейс IDisposable для гарантированного освобождения неуправляемых ресурсов. При удалении игрового объекта или выгрузке сцены соответствующие вызовы `glDeleteBuffers`, `glDeleteTextures` и `glDeleteProgram` выполняются автоматически. Весь код, связанный с OpenGL, изолирован в отдельном модуле графической подсистемы, что позволяет в будущем заменить OpenGL на другой графический API (например, Vulkan) без изменения остальных компонентов движка. Таким образом, выбор OpenGL 4.6 обеспечивает баланс между функциональностью, совместимостью, производительностью и сложностью

разработки, полностью соответствующий целям и ограничениям дипломного проекта.

3.1.3. Windows Forms для реализации редактора сцен

Для реализации графического интерфейса редактора сцен выбрана технология Windows Forms. Данный выбор обусловлен простотой разработки, нативной интеграцией с операционной системой Windows и тесной совместимостью с языком C#, который используется для разработки основного движка. Windows Forms является зрелой и хорошо документированной технологией, предоставляющей разработчику богатую коллекцию стандартных элементов управления, таких как кнопки, текстовые поля, списки, чекбоксы и панели, что позволяет быстро создавать функциональный интерфейс без необходимости написания сложного кода для отрисовки элементов.

Основным преимуществом Windows Forms для целей данной работы является простота интеграции с трёхмерной визуализацией на основе OpenGL. Windows Forms предоставляет элемент Panel, внутрь которого можно встроить окно OpenGL через получение дескриптора окна (HWND) и передачу его в библиотеку GLFW или непосредственно в OpenGL. Это позволяет размещать область трёхмерного предпросмотра сцены непосредственно внутри окна редактора, наряду с панелями инструментов и элементами управления. Альтернативные технологии, такие как WPF, предоставляют более современные возможности кастомизации интерфейса, но требуют значительно больше усилий для интеграции с нативным рендерингом OpenGL из-за особенностей своей композиции и использования DirectX. В контексте дипломной работы, где приоритетом является создание работающего прототипа, а не излишне сложного интерфейса, Windows Forms является наиболее прагматичным выбором.

Архитектурно редактор сцен на Windows Forms представляет собой главное окно, содержащее следующие основные элементы управления. Панель трёхмерного предпросмотра отображает текущую сцену в режиме реального времени, используя ту же графическую подсистему OpenGL, что и

игровой движок. Кнопка импорта модели открывает стандартный диалог выбора файла и запускает процесс загрузки STL или GLB модели. Текстовые поля для настройки камеры редактора позволяют изменять чувствительность управления, скорость перемещения, ближнюю и дальнюю дистанции отсечения. Список превью отображает все загруженные в сцену игровые объекты в виде миниатюр или текстовых названий, обеспечивая быстрый доступ к любому объекту для его выбора и редактирования. Отдельное окно инспектора объекта открывается при выборе объекта в сцене или в списке превью и содержит поля для редактирования трансформации объекта, чекбокс видимости в игровом режиме, поле для задания тега, а также кнопки удаления объекта и возврата к главному окну.

Важной особенностью реализации является разделение режимов работы редактора. В режиме редактирования пользователь может свободно перемещать камеру, выбирать объекты и изменять их параметры. В режиме игры редактор скрывает элементы управления и передаёт управление игровой камере, параметры которой были настроены разработчиком. Переключение между режимами осуществляется по нажатию соответствующей кнопки или сочетания клавиш. Такое разделение позволяет разработчику игр тестировать сцену в условиях, приближенных к реальному игровому процессу, не выходя из редактора.

Обработка событий в Windows Forms организована через стандартную событийную модель. При нажатии кнопки импорта модели вызывается соответствующий обработчик, который запускает процедуру загрузки файла и добавления объекта в сцену. При изменении значения в текстовом поле обновляется соответствующий параметр камеры или трансформации объекта. При клике по объекту в трёхмерной области предпросмотра выполняется ray casting для определения выбранного объекта и открывается инспектор. Все события обрабатываются в основном потоке пользовательского интерфейса, а ресурсоёмкие операции, такие как импорт больших моделей, выносятся в асинхронные задачи с отображением индикатора загрузки, чтобы не блокировать интерфейс.

Таким образом, выбор Windows Forms для реализации редактора сцен обусловлен оптимальным соотношением скорости разработки, функциональности и простоты интеграции с OpenGL-рендерингом. Технология позволяет создать полноценный инструмент для работы со сценами, не отвлекаясь на низкоуровневую реализацию элементов управления, что особенно важно в рамках дипломного проекта с ограниченными временными ресурсами.

3.1.4. Библиотека SixLabors.ImageSharp

Для загрузки и обработки текстурных изображений в разрабатываемом игровом движке используется библиотека SixLabors.ImageSharp. Данная библиотека представляет собой полностью управляемую, кроссплатформенную и высокопроизводительную библиотеку для работы с двухмерной графикой на языке C#. ImageSharp поддерживает широкий спектр форматов изображений, включая PNG, JPEG, BMP, GIF, TIFF, а также современный формат WebP. В отличие от нативных решений, библиотека не имеет внешних зависимостей и полностью написана на управляемом коде, что упрощает распространение приложения и исключает необходимость установки дополнительных системных библиотек.

В контексте игрового движка ImageSharp используется для загрузки текстурных файлов, их декодирования и преобразования в формат, пригодный для передачи в OpenGL. Библиотека предоставляет простой и интуитивно понятный API, позволяющий загрузить изображение вызовом метода Image.Load, после чего получить доступ к пиксельным данным. Полученный массив пиксельных данных может быть передан в OpenGL через вызов функции glTexImage2D для создания текстурного объекта. Благодаря использованию механизмов SIMD, ImageSharp демонстрирует высокую производительность, приближающуюся к нативным решениям, что особенно важно при загрузке большого количества текстур при старте сцены.

При загрузке изображений, размер которых не превышает определённого порогового значения, библиотека гарантирует непрерывность пиксельных данных в памяти. Эта особенность позволяет передавать указатель

на данные непосредственно в OpenGL без дополнительного копирования, что повышает эффективность работы с видеопамью. Библиотека распространяется под лицензией Apache 2.0, что делает её свободно используемой в некоммерческих и образовательных проектах. Таким образом, выбор SixLabors.ImageSharp обусловлен её функциональностью, производительностью и отсутствием нативных зависимостей, что полностью соответствует требованиям разрабатываемого игрового движка.

3.2. Общая структура движка

Игровой движок состоит из следующих основных модулей: графической подсистемы на базе OpenGL, системы управления игровыми объектами и компонентами, менеджера сцен, системы камер и освещения, редактора сцен на Windows Forms, системы импорта моделей (STL, GLB), подсистемы обработки ввода и вспомогательных утилит. Графическая подсистема взаимодействует со сценой для получения отображаемых объектов. Редактор сцен через менеджер игровых объектов сцены управляет составом сцены и через графическую подсистему отображает трёхмерный предпросмотр. Система импорта моделей создает объекты и добавляет их в сцену через менеджер объектов. На рисунке 3.1 представлена общая схема архитектуры программной системы.

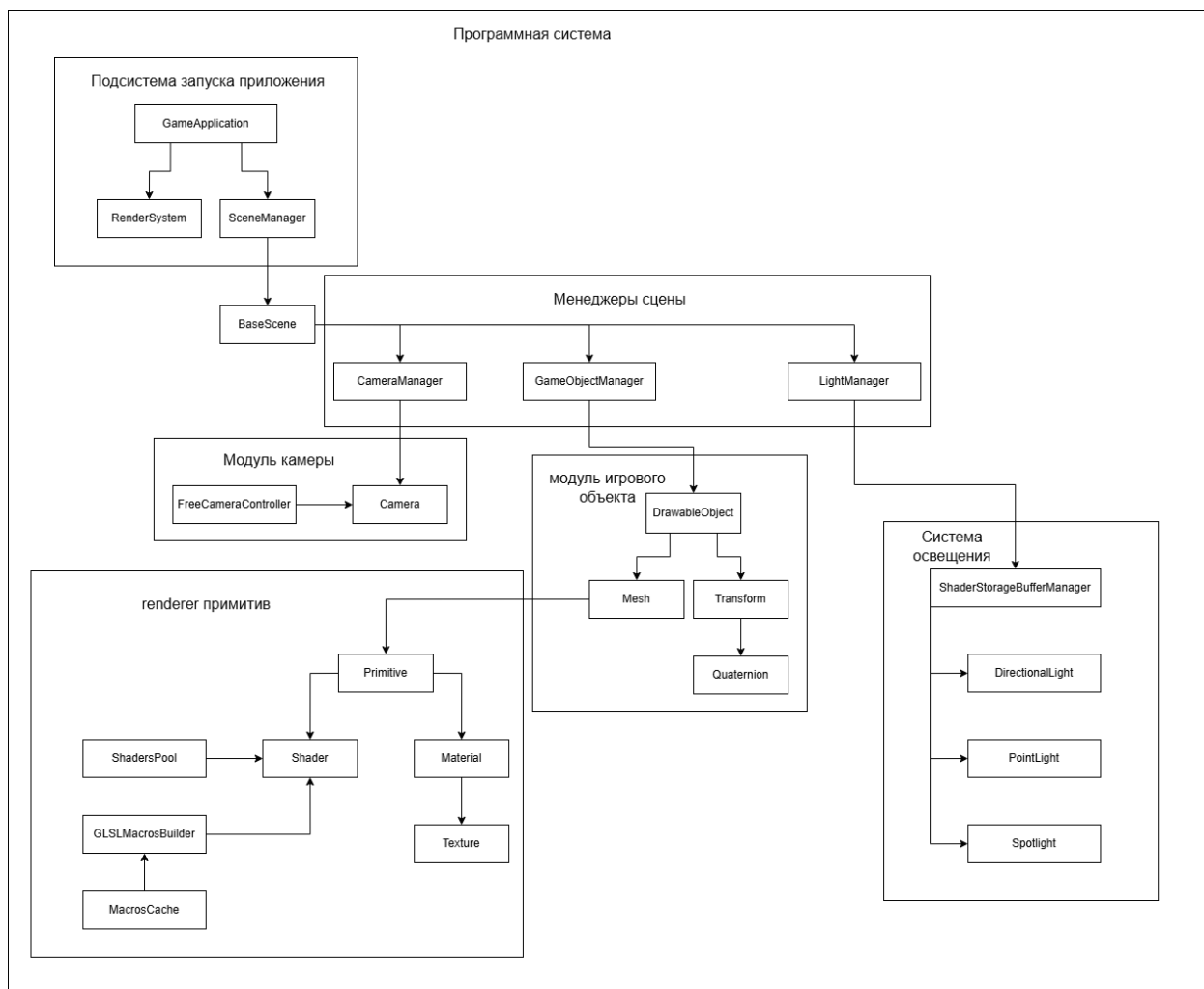


Рисунок 3.1 – Общая схема архитектуры приложения

Данная программная система содержит множество компонентов, взаимодействующих между собой. В таблице 3.1 представлены основные компоненты системы и их назначение.

Таблица 3.1 – Назначение компонентов программной системы

Имя	Описание
Camera	Данный класс представляет собой камеру.
FreeCameraController	Контроллер камеры. Поддерживает движение камеры и вращение камеры в углах yaw (рысканье) и pitch (тангаж).
Material	Материал, который создается при загрузке моделей.

Продолжение таблицы 3.1

Имя	Описание
Mesh	Меш модели. Выступает контейнером для примитивов и содержит матрицу в случае .glb модели.
Primitive	Примитив геометрии. Содержит буферы и шейдер.
Texture	Текстура. Является составной частью материала.
Transform	Компонент игрового объекта. Управляет его положением, поворотом и масштабом.
DrawableObject	Базовый класс для игровых объектов.
GLBGameObject	Игровой объект, созданный на основе .glb файла.
STLGameObject	Игровой объект, созданный на основе .stl файла.
DirectionalLight	Является направленным источником освещения.
PointLight	Является точечным источником освещения.
Spotlight	Является прожекторным источником освещения.
CameraManager	Менеджер камер. Управляет камерами в сцене.
GameObjectManager	Менеджер игровых объектов. Управляет игровыми объектами в сцене.
LightManager	Менеджер света. Управляет источниками освещения в сцене.
SceneManager	Менеджер сцен. Управляет загрузкой и выгрузкой сцен.

Продолжение таблицы 3.1

Имя	Описание
GLSLMacrosBuilder	Вспомогательный класс для шейдера. Валидирует и добавляет в шейдер макросы на основе параметров рендеринга.
MacrosCache	Хранит названия макросов, соотнесенные с настройками рендеринга.
TextureActivator	Активирует текстуры для шейдера. Хранит кэш последних привязанных текстур.
OffScreenRender	Необходим для работы визуального редактора. Рисует объекты в текстуры.
RenderSystem	Система рендеринга. Отрисовывает объекты сцены.
Shader	Шейдер. Хранится в примитиве.
ShadersPool	Пул шейдеров. Необходим для кэширования шейдеров одинаковых материалов.
Mathematics	Хранит в себе различные математические операции, используемые движком.
Quaternion	Кватернион. Используется для расчета поворота объекта.
BaseScene	Базовый класс сцены.
GameApplication	Главный класс движка. Управляет запуском приложения.
SystemCalls	Используется движком для вывода диагностических сообщений в консоль.
GLBImporter	Импортер .glb моделей.
GLBModel	GLB модель. Используется для создания GLBGameObject.
GLBScene	Контейнер для GLB моделей.
Node	Узел иерархии узлов GLB файла.

Продолжение таблицы 3.1

Имя	Описание
STLImporter	Импортер .stl моделей.
STLModel	STL модель. Используется для создания STLGameObject.

3.3. Взаимодействие основных модулей

Основные модули движка взаимодействуют следующим образом. Редактор сцен инициирует операции (импорт модели, сохранение сцены, настройка параметров игрового объекта), обращаясь к менеджеру сцен и графической подсистеме. Менеджер сцен управляет составом сцены и жизненным циклом объектов. Графическая подсистема запрашивает данные о сцене у менеджера сцен и выполняет рендеринг. Подсистема ввода передаёт события через сцену к игровым объектам. Диаграмма последовательности на рисунке 3.2 демонстрирует порядок вызовов при рендеринге игрового объекта.

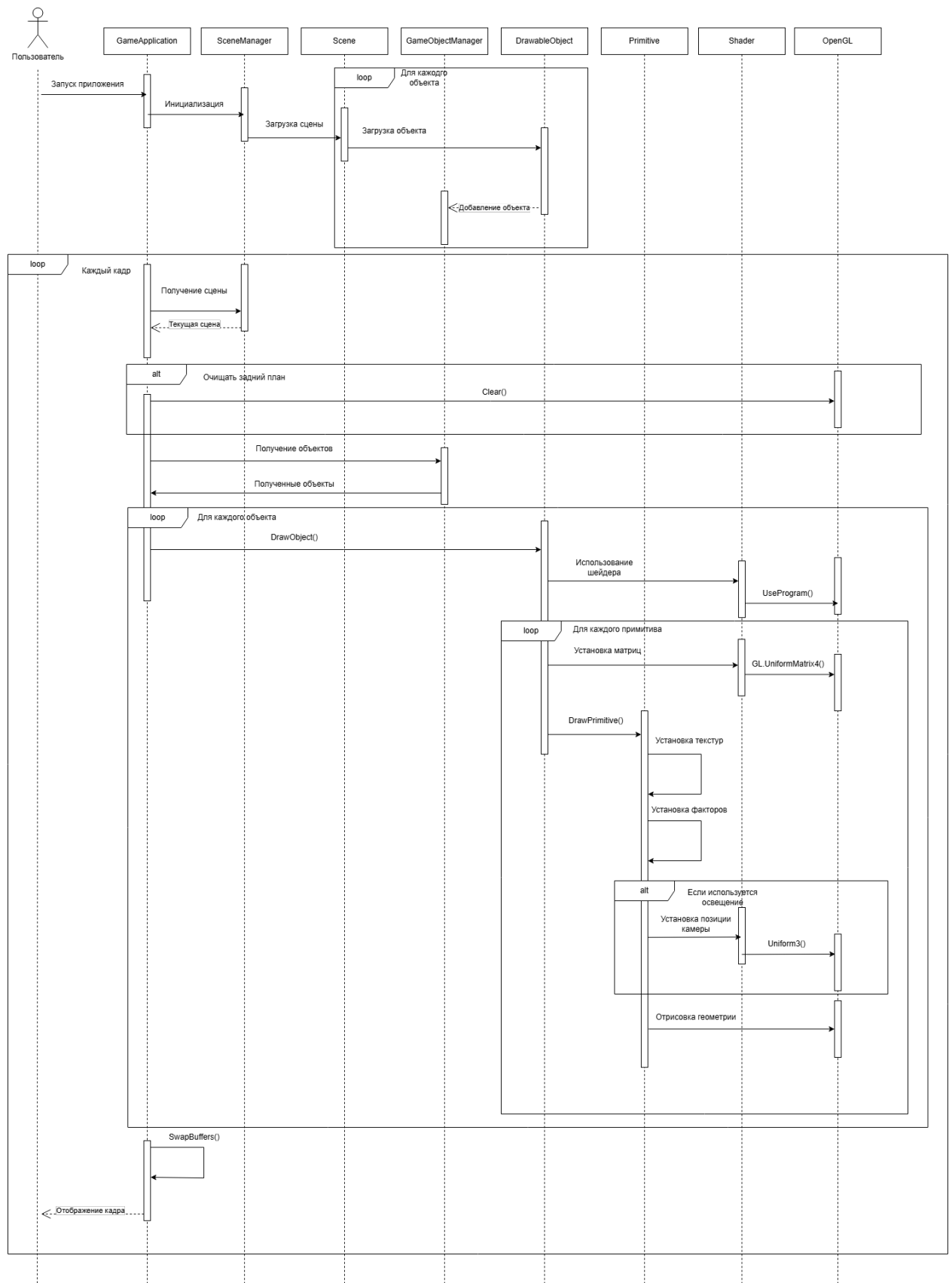


Рисунок 3.2 – Диаграмма последовательности для цикла рендеринга объекта

3.4. Графическая подсистема

3.4.1. Управление буферами вершин и индексов

Для хранения геометрических данных в движке используются вершинные буферы (VBO) и индексные буферы (EBO). Вершинный буфер хранит массив вершин, каждая из которых содержит позицию (три координаты), нормаль (три компонента) и текстурные координаты (две координаты). Индексный буфер хранит последовательность индексов, определяющих порядок соединения вершин в треугольники. Все буферы инкапсулированы в классе `Primitive`. При создании примитива в игровом объекте выполняются следующие операции: генерация идентификаторов буферов; привязка буфера к целевой точке; загрузка данных в буфер; настройка атрибутов вершин и активация атрибутов через. Для организации атрибутов используется объект вершинного массива (VAO), который сохраняет все настройки атрибутов и привязки буферов. Это позволяет переключаться между разными моделями. При уничтожении меша буферы удаляются через `glDeleteBuffers`, что гарантирует освобождение памяти видеокарты.

3.4.2. Компиляция и загрузка шейдерных программ

В разработанном движке шейдерные программы собираются динамически на основе набора макросов, определяющих характеристики материала, модель освещения и глобальные настройки рендеринга. Класс `Shader` инкапсулирует процесс создания шейдерной программы: создаёт идентификатор программы, компилирует вершинный и фрагментный шейдеры, линкует их и сохраняет соответствие имён `uniform`-переменных их местоположениям.

Перед компиляцией шейдера класс `GLSLMacrosBuilder` формирует строку с макросами, которая вставляется в начало исходного кода шейдера. Для этого используются атрибуты `GLSLMacros`, которыми помечены элементы перечислений `RenderSettings` и `ShadingModels`. Это позволяет включать в шейдер только те участки кода, которые необходимы для текущего материала: определённые текстуры, факторы материалов или модели освеще-

ния. Глобальные настройки рендеринга (максимальная интенсивность бликов, сила окружающего освещения, использование монохромного окружающего света) передаются в шейдер через макросы, сформированные на основе класса `GlobalRenderSettings`. При необходимости в шейдер также включается макрос наличия матрицы меша.

После компиляции шейдер загружается в видеопамять. В случае ошибки компиляции или линковки движок выводит в консоль лог ошибок. При включённом режиме `DEBUG_MODE` скомпилированные шейдеры сохраняются в текстовые файлы для отладки. В таблицах 3.2 и 3.3 представлены поддерживаемые флаги рендеринга и модели освещения, а в таблице 3.4 — глобальные настройки.

Таблица 3.2 – Render settings

Имя	Описание
<code>VertexColors</code>	Цвета вершин.
<code>AlbedoMap</code>	Основная текстура материала.
<code>MetallicRoughnessMap</code>	Карта металличности/шероховатости.
<code>NormalMap</code>	Карта нормалей.
<code>OcclusionMap</code>	Карта оклюзий.
<code>EmissiveMap</code>	Карта эмиссии.
<code>BaseColorFactor</code>	Общий цвет полигонов.
<code>MetallicFactor</code>	Значение металличности.
<code>RoughnessFactor</code>	Значение шероховатости.
<code>EmissiveFactor</code>	Значение эмиссии.

Таблица 3.3 – Модели освещения

Имя	Описание
<code>BlinnPhong</code>	Модель освещения Блинна-Фонга.
<code>Gouraud</code>	Модель освещения Гуро.

Продолжение таблицы 3.3

Имя	Описание
PBR MetallicRoughness	Модель освещения для physically based rendering материалов. Не используется.
Unlit	Отсутствие освещения.

Таблица 3.4 – Модели освещения

Имя	Описание
MaxShininess	Максимальная интенсивность зеркальных бликов (значение по умолчанию — 32)
AmbientStrength	Сила окружающего освещения (значение по умолчанию — 0.1)
UseMonochromeAmbient	Флаг использования монохромного окружающего освещения вместо цветного

3.5. Система трансформации

Система трансформации отвечает за управление положением, поворотом и масштабом игровых объектов в трёхмерном пространстве. Каждый игровой объект содержит компонент трансформации, который хранит его параметры в мировом пространстве.

3.5.1. Матрицы преобразований

Для преобразования вершин объекта из локальных координат в мировые используется матрица модели (model matrix). Эта матрица получается путём последовательного перемножения матриц масштабирования, поворота и перемещения. Формула вычисления матрицы модели имеет следующий вид:

$$\mathbf{M} = \mathbf{T} \times \mathbf{R} \times \mathbf{S}$$

где **M** — итоговая матрица модели, **T** — матрица перемещения, **R** — матрица поворота, **S** — матрица масштабирования.

Матрица перемещения имеет следующий вид:

$$\mathbf{T} = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

где t_x, t_y, t_z — координаты перемещения объекта в мировом пространстве.

Матрица масштабирования имеет следующий вид:

$$\mathbf{S} = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

где s_x, s_y, s_z — коэффициенты масштабирования по осям X, Y и Z соответственно.

В движке матрицы хранятся в виде указателей на массив чисел с плавающей точкой (float*), что позволяет эффективно передавать их в OpenGL без дополнительного копирования данных. Все матричные операции — умножение, транспонирование, вычисление обратной матрицы — выполняются статическими методами класса Mathematics. Для отображения объекта на экране матрица модели последовательно умножается на матрицу вида камеры и матрицу проекции, формируя итоговую матрицу MVP:

$$\mathbf{MVP} = \mathbf{P} \times \mathbf{V} \times \mathbf{M}$$

где **P** — матрица проекции, **V** — матрица вида, **M** — матрица модели. Эта матрица передаётся в вершинный шейдер и применяется к каждой вершине объекта.

3.5.2. Позиция, поворот и масштаб

Компонент трансформации хранит три основных параметра в мировом пространстве: позицию, поворот и масштаб. Позиция задаётся вектором из трёх координат, определяющим смещение объекта в пространстве. Поворот задаётся кватернионом — математическим объектом, представляющим вращение в трёхмерном пространстве. Масштаб задаётся вектором из трёх коэффициентов, позволяя независимо масштабировать объект по каждой оси.

Кватернион в движке реализован собственным классом и представляет собой четырёхкомпонентный вектор вида:

$$\mathbf{q} = (x, y, z, w) = \sin\left(\frac{\theta}{2}\right) \cdot \mathbf{u} + \cos\left(\frac{\theta}{2}\right)$$

где θ — угол поворота, $\mathbf{u}$ — единичный вектор оси вращения. Использование кватернионов вместо углов Эйлера позволяет избежать явления блокировки шарнирного соединения и обеспечивает плавную интерполяцию вращений.

Преобразование кватерниона в матрицу поворота выполняется по следующей формуле:

$$\mathbf{R} = \begin{pmatrix} 1 - 2y^2 - 2z^2 & 2xy - 2zw & 2xz + 2yw & 0 \\ 2xy + 2zw & 1 - 2x^2 - 2z^2 & 2yz - 2xw & 0 \\ 2xz - 2yw & 2yz + 2xw & 1 - 2x^2 - 2y^2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

где (x, y, z, w) — компоненты кватерниона.

Компонент трансформации предоставляет поля для хранения позиции (`Vector3`), поворота (`Quaternion`) и масштаба (`Vector3`). При изменении любого из этих параметров пересчитывается матрица модели через вызов методов класса `Mathematics`. Матрица модели хранится в виде массива из шестнадцати чисел с плавающей точкой, готового к передаче в OpenGL. Для удобства работы с поворотом также доступны методы для установки вращения через

углы Эйлера, которые внутренне преобразуются в кватернион с использованием соответствующих математических функций.

3.6. Система камер

Система камер отвечает за формирование изображения сцены с определённой точки обзора. В разработанном движке камера реализована классом `Camera`, который хранит параметры проекции, положение и ориентацию в пространстве.

3.6.1. Типы проекций (перспективная и ортографическая)

В движке поддерживаются два типа проекций. Перспективная проекция используется в игровом режиме и характеризуется углом обзора (FOV) и соотношением сторон экрана.

3.6.2. Управление положением и направлением обзора

Положение камеры задаётся вектором `Position`. Направление обзора определяется тремя векторами: `Front` (направление взгляда), `RightVector` (правая сторона) и `Up` (верх). Управление направлением осуществляется через углы рыскания (`Yaw`) и тангажа (`Pitch`). При их изменении вызывается метод `CalculateVectors`, который пересчитывает векторы `Front`, `RightVector` и `Up`.

3.6.3. Матрица вида и матрица проекции

Матрица вида (`View Matrix`) преобразует мировые координаты в координаты пространства камеры. В движке она вычисляется как произведение матрицы ориентации и матрицы перемещения.

Матрица ориентации строится из векторов `RightVector`, `Up` и `Front`:

$$\mathbf{R} = \begin{pmatrix} r_x & r_y & r_z & 0 \\ u_x & u_y & u_z & 0 \\ -f_x & -f_y & -f_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

где (r_x, r_y, r_z) — компоненты **RightVector**, (u_x, u_y, u_z) — компоненты **Up**, (f_x, f_y, f_z) — компоненты **Front**.

Матрица перемещения строится из позиции камеры:

$$\mathbf{T} = \begin{pmatrix} 1 & 0 & 0 & -p_x \\ 0 & 1 & 0 & -p_y \\ 0 & 0 & 1 & -p_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

где (p_x, p_y, p_z) — компоненты **Position**.

Итоговая матрица вида вычисляется как произведение:

$$\mathbf{V} = \mathbf{R} \times \mathbf{T}$$

Матрица проекции (**Projection Matrix**) преобразует координаты из пространства камеры в экранные координаты. Для перспективной проекции матрица имеет следующий вид:

$$\mathbf{P}_{persp} = \begin{pmatrix} \frac{1}{\text{aspect} \cdot \tan(fov/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(fov/2)} & 0 & 0 \\ 0 & 0 & -\frac{Far+Near}{Far-Near} & -\frac{2 \cdot Far \cdot Near}{Far-Near} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

Для ортогографической проекции матрица имеет следующий вид:

$$\mathbf{P}_{ortho} = \begin{pmatrix} \frac{2}{right-left} & 0 & 0 & -\frac{right+left}{right-left} \\ 0 & \frac{2}{top-bottom} & 0 & -\frac{top+bottom}{top-bottom} \\ 0 & 0 & -\frac{2}{Far-Near} & -\frac{Far+Near}{Far-Near} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

В движке обе матрицы хранятся в виде указателей на массивы из шестнадцати чисел с плавающей точкой в неуправляемой памяти. Матрица вида заполняется через вызов `Mathematics.MultiplyMatrices4`. Матрица проекции заполняется напрямую в зависимости от типа камеры.

3.7. Система освещения

3.7.1. Типы источников света (точечные, направленные, окружающие)

В разработанном игровом движке реализована система освещения, поддерживающая три типа источников света, каждый из которых имеет свои особенности и область применения.

Окружающее освещение (Ambient Light) представляет собой базовое равномерное освещение всей сцены, не зависящее от положения источников и направления лучей. Этот тип освещения имитирует рассеянный свет, многократно отражённый от поверхностей объектов и стен. Окружающее освещение не создаёт теней и не подчёркивает объём объектов, но обеспечивает видимость всех элементов сцены даже в отсутствие прямых источников. В движке окружающее освещение задаётся двумя параметрами: параметром — `AmbientStrength`, который задает интенсивность освещения и параметром `UseMonochromeAmbient`, который задает цвет амбиентного освещения. Если он выставлен в `true`, то освещение будет серым вне зависимости от цвета объекта. в ином случае данное освещение будет цвета объекта. Окружающее освещение применяется ко всем объектам сцены глобально.

Точечный источник света (Point Light) излучает свет равномерно во все стороны из одной точки в пространстве, подобно обычной лампочке. Такой

источник характеризуется положением в мировых координатах, цветом, интенсивностью и радиусом действия, который задается параметрами Constant, Linear и Quadratic. Интенсивность освещения от точечного источника убывает с расстоянием по квадратичному закону, что создаёт естественный эффект затухания. В движке точечные источники могут быть добавлены в сцену в любом количестве, каждый из них независимо освещает окружающие объекты. Точечные источники подходят для имитации ламп, факелов, взрывов и других локализованных источников света.

Направленный источник света (Directional Light) имитирует бесконечно удалённый источник, такой как Солнце. В отличие от точечного, направленный источник не имеет позиции в пространстве, а характеризуется только направлением лучей. Все лучи направленного источника параллельны друг другу, поэтому освещённость объекта не зависит от расстояния до источника. В движке направленный источник задаётся углами Эйлера Yaw и Pitch, цветом и интенсивностью. Этот тип освещения используется для создания дневного освещения на открытых пространствах.

Для всех типов источников света в шейдере реализована модель освещения Фонга, включающая вычисление рассеянной (diffuse) и зеркальной (specular) составляющих для точечных и направленных источников. Окружающее освещение добавляется на финальном этапе как константная составляющая, не зависящая от геометрии сцены.

Прожектор (Spotlight) Четвёртым типом источника света, реализованным в движке, является прожектор. Прожектор представляет собой точечный источник, излучающий свет не во все стороны, а в пределах заданного конуса. Такой тип освещения имитирует направленные источники света: фонарик, автомобильную фару, уличный прожектор или луч света на сцене.

Прожектор характеризуется следующими параметрами: положение в мировых координатах, определяющее точку испускания света; Углы Эйлера Yaw и Pitch, задающие ось конуса; цвет и интенсивность излучения; угол внутреннего конуса, внутри которого свет достигает максимальной яркости; угол внешнего конуса, за пределами которого освещение полностью отсут-

ствует; а также радиус действия, ограничивающий максимальное расстояние распространения света.

3.7.2. Модель освещения

В разработанном игровом движке реализованы две модели освещения: модель Гуро и модель Блинна-Фонга, переключение между которыми выполняется через параметр шейдера.

Модель освещения Гуро вычисляет интенсивность освещения в вершинах многоугольника с последующей линейной интерполяцией полученных значений по всей поверхности грани. В вершинном шейдере для каждой вершины рассчитываются цветовые компоненты: окружающая, рассеянная и зеркальная. Полученные цвета интерполируются растеризатором, и для каждого фрагмента используется уже вычисленное значение. Основным преимуществом модели Гуро является высокая производительность, поскольку вычисления освещения выполняются только для вершин, а не для каждого пикселя. Недостатком является заметная ступенчатость освещения на крупных полигонах, особенно в области зеркальных бликов.

Модель освещения Блинна-Фонга вычисляет освещение для каждого фрагмента независимо. Во фрагментный шейдер передаются нормаль, позиция и текстурные координаты, интерполированные из вершинных атрибутов. Для каждого фрагмента рассчитываются три составляющие итогового цвета.

Формула модели Блинна-Фонга имеет следующий вид:

$$I = I_{ambient} + I_{diffuse} + I_{specular}$$

где $I_{ambient}$ — компонента окружающего освещения:

$$I_{ambient} = k_a \cdot I_a$$

Компонента рассеянного освещения (диффузная):

$$I_{diffuse} = k_d \cdot I_d \cdot \max(0, \vec{L} \cdot \vec{N})$$

где k_d — коэффициент диффузного отражения материала, I_d — интенсивность источника света, $\vec{L}$ — единичный вектор от точки поверхности к источнику света, $\vec{N}$ — нормаль к поверхности.

Компонента зеркального отражения по модели Блинна-Фонга:

$$I_{\text{specular}} = k_s \cdot I_s \cdot (\vec{N} \cdot \vec{H})^n$$

где k_s — коэффициент зеркального отражения, I_s — интенсивность зеркальной составляющей источника, $\vec{H}$ — половинный вектор, вычисляемый как $\vec{H} = \frac{\vec{L} + \vec{V}}{|\vec{L} + \vec{V}|}$, $\vec{V}$ — вектор направления на камеру, n — показатель степени, определяющий размер блика (коэффициент шероховатости поверхности).

Основным отличием модели Блинна-Фонга от классической модели Фонга является использование половинного вектора $\vec{H}$ вместо вычисления вектора отражения, что упрощает вычисления и даёт более реалистичные блики.

В разработанном движке модель освещения поддерживает до 8 источников света одновременно, каждый из которых может быть точечным, направленным или прожектором. Окружающее освещение задаётся глобально для всей сцены. Все коэффициенты освещения (диффузный, зеркальный, интенсивность) передаются в шейдер через uniform-переменные и могут изменяться в реальном времени через редактор материалов.

3.7.3. Работа с SSBO

Для эффективной передачи данных о источниках света в шейдерную программу в движке используется механизм Shader Storage Buffer Object. SSBO представляет собой буфер, доступный для чтения и записи как со стороны графического процессора, так и со стороны центрального процессора, что позволяет хранить в видеопамяти структурированные массивы данных произвольного размера. На рисунке 3.3 показана схема хранения данных света в SSBO на примере направленного источника света.

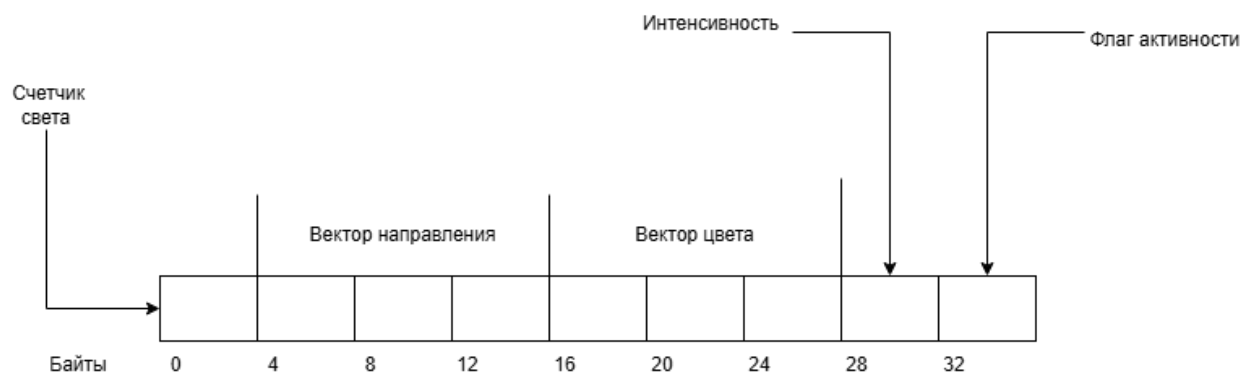


Рисунок 3.3 – Схема хранения данных в памяти

В движке реализован обобщённый класс `ShaderStorageBufferManager<T>`, параметризуемый типом структуры, описывающей данные источника света. Данный класс инкапсулирует все операции по управлению буфером: выделение памяти, добавление, обновление и удаление элементов, а также автоматическое расширение буфера при превышении начальной ёмкости.

При инициализации менеджера в конструктор передаются начальная ёмкость буфера и точка привязки (binding point). Внутри конструктора выполняется выделение памяти в оперативной памяти через `NativeMemory.Alloc` и в видеопамяти через вызов `glGenBuffer` и `glBufferData`. Для хранения количества активных источников в начале буфера отводится дополнительное пространство размером `PADDING`, что позволяет шейдеру знать, сколько элементов обрабатывать. Указатель на выделенную память хранится в поле `_lightData`, а идентификатор буфера OpenGL — в свойстве `LightHandler`.

Метод `AddLight` добавляет новый источник света в буфер. При добавлении проверяется, не превышает ли текущее количество активных источников заявленную ёмкость. При превышении ёмкости буфер автоматически расширяется в два раза через вызов `NativeMemory.Realloc`, после чего обновляются данные в видеопамяти. При наличии свободных мест (отслеживаемых через поле `_freeSpaces`) новый источник размещается на позиции ранее удалённого элемента. Это позволяет избежать фрагментации данных и пересортировки массива. После добавления обновляется счётчик активных источников в начале буфера.

Метод `RemoveLight` выполняет логическое удаление источника света, устанавливая поле `isActive` в соответствующей структуре в ноль. Счётчик активных источников уменьшается, а счётчик свободных мест увеличивается. Физического удаления данных из буфера не происходит, что позволяет избежать дорогостоящего сдвига массива и сохраняет производительность при частых удалениях.

Метод `UpdateLight` обновляет данные существующего источника по указанному индексу. Вычисляется смещение в буфере как произведение размера структуры на индекс плюс отступ `PADDING`, после чего через `Marshal.StructureToPtr` данные копируются в управляемую память, а затем через `glBufferSubData` — в видеопамять.

Метод `Clear` полностью очищает буфер, устанавливая счётчик активных источников в ноль и перезаписывая данные в видеопамяти через `glBufferData`.

Все операции с видеопамятью выполняются через привязку буфера к цели `ShaderStorageBuffer` с последующей отвязкой. Буфер также привязывается к указанной точке привязки через `glBindBufferBase`, что позволяет шейдеру обращаться к данным по этой точке.

Использование SSBO для хранения источников света даёт следующие преимущества: отсутствует ограничение на максимальное количество источников, характерное для `uniform`-массивов; данные могут динамически изменяться в рантайме без перекомпиляции шейдера; массив структур хранится в памяти GPU как непрерывный блок, что обеспечивает эффективный доступ из шейдера. В разработанном движке SSBO управляет массивами точечных источников, направленных источников и прожекторов, каждый из которых описывается отдельной структурой.

3.8. Управление сценами

3.8.1. Загрузка и выгрузка сцен

Менеджер сцен обеспечивает управление жизненным циклом игровых сцен, включая их загрузку, выгрузку, переключение и организацию истории переходов. Все загруженные сцены регистрируются в реестре по имени, что позволяет обращаться к ним без необходимости хранения прямых ссылок.

Загрузка сцены. При загрузке сцены менеджер сначала проверяет, не является ли запрашиваемая сцена уже текущей, чтобы избежать повторной инициализации. Если текущая сцена существует, для неё вызывается метод выхода, который приостанавливает игровую логику и уведомляет подписанные компоненты о завершении работы сцены. Затем выполняется очистка ресурсов текущей сцены через метод `Cleanup`, который освобождает все занятые объектом ресурсы: шейдерные программы, VBO, EBO, VAO, текстурные объекты, источники света и камеры. После очистки новая сцена устанавливается как текущая, ей передаётся ссылка на менеджер сцен, после чего выполняется десериализация сцены из файла, восстанавливающая все игровые объекты, их компоненты и параметры. Если приложение находится не в режиме редактирования, для новой сцены вызывается метод старта, иницирующий игровую логику. По завершении загрузки генерируются события, оповещающие другие подсистемы движка об изменении текущей сцены.

Выгрузка сцены. При выгрузке текущая сцена удаляется из памяти. Если приложение не в режиме редактирования, сначала вызывается метод выхода, дающий сцене возможность корректно завершить игровые процессы. Затем выполняется метод `Cleanup`, который рекурсивно обходит все игровые объекты сцены и для каждого из них освобождает связанные ресурсы: удаляются шейдерные программы, уничтожаются вершинные и индексные буферы через вызов соответствующих функций OpenGL, освобождаются текстурные объекты, очищаются массивы источников света в SSBO-буферах, а также освобождаются указатели камер и других компонентов. После завершения очистки ссылка на текущую сцену обнуляется, и генерируются собы-

тия об изменении состояния. Менеджер сцен не удаляет саму сцену из реестра, что позволяет загрузить её повторно в будущем без повторной регистрации.

Стек сцен. Менеджер поддерживает стек сцен, позволяющий временно переключаться между сценами с возможностью возврата. Метод `PushScene` сохраняет текущую сцену в стеке и загружает новую. Метод `PopScene` выгружает текущую сцену и восстанавливает сцену из вершины стека. Этот механизм полезен для реализации наложения экранов, например, при открытии меню паузы поверх игровой сцены.

Пауза сцены. Менеджер также управляет состоянием паузы текущей сцены. При установке флага паузы вызываются соответствующие методы сцены, приостанавливающие обновление игровой логики без выгрузки ресурсов. Возобновление работы сцены происходит при снятии паузы.

Полная очистка. При завершении работы приложения или при необходимости полного сброса состояния вызывается метод `Cleanup`, который очищает все сцены в стеке, реестре и текущую сцену, а также отписывает все события. Этот метод гарантирует, что все неуправляемые ресурсы OpenGL будут освобождены до закрытия приложения, предотвращая утечки памяти.

3.8.2. Сохранение сцены в файл и восстановление

Для обеспечения возможности сохранения и последующей загрузки игровых сцен в движке реализован механизм сериализации в JSON-формат. Каждая сцена сохраняется в отдельный файл, имя которого соответствует имени класса сцены, что позволяет однозначно идентифицировать сцену при загрузке.

Формат файла сцены. Файл сцены имеет JSON-структуру, включающую два основных массива: массив путей к файлам моделей и массив описаний игровых объектов. Пример структуры сохранённой сцены представлен ниже:

```

1  {
2    "Models": [
3      "C:\\Users\\it_ge\\Desktop\\Amethyst-game-engine\\Client\\bin\\Debug\\net10.0-windows10
4      .0.17763.0\\Models\\Room for debug.glb"
5    ],
6    "GameObjects": [
7      {
8        "Tag": "Untagged",
9        "Type": "GLB",
10       "ModelIndex": 0,
11       "GLBModelIndex": 0,
12       "Transform": {
13         "Position": [0, 0, 0],
14         "Rotation": [0, 0, 0],
15         "Scale": [1, 1, 1]
16       }
17     ]
18   }

```

Listing 1 – Пример JSON-файла сохранённой сцены

Массив путей к моделям содержит абсолютные пути к файлам импортированных моделей в форматах STL и GLB. Это позволяет при десериализации однократно загрузить каждую модель и использовать её для создания нескольких игровых объектов без дублирования данных. Массив игровых объектов содержит для каждого объекта следующие параметры: строковый тег для идентификации объекта, тип модели (STL или GLB), индекс модели в массиве путей, для GLB-моделей также индекс модели внутри составного файла, а также объект трансформации, включающий позицию, поворот и масштаб по трём осям. Позиция, поворот и масштаб хранятся в виде массивов из трёх чисел с плавающей точкой.

Сериализация сцены. Процесс сохранения сцены инициируется вызовом метода `SerializeScene`. В начале работы метода проверяется наличие папки `Saves` в директории приложения; при отсутствии папка создаётся автоматически. Затем формируется JSON-объект сцены: создаются пустые массивы для моделей и игровых объектов. Менеджер игровых объектов обходит для сбора всех используемых моделей, пути к которым добавляются в массив моделей. Далее каждый игровой объект преобразуется в JSON-объект с заполнением его тега, типа, индекса модели и параметров трансформации. Для объектов, импортированных из GLB, дополнительно сохраняется индекс под-

сцены внутри файла. Сформированный JSON-объект записывается в файл с отступами для удобства чтения и отладки.

Десериализация сцены. Восстановление сцены выполняется методом `DeserializeScene`. Сначала определяется полный путь к файлу сцены на основе имени текущего класса сцены. Если файл не существует, загрузка не выполняется и сцена остаётся пустой. При наличии файла происходит чтение и парсинг JSON-документа. Из корневого элемента извлекается массив путей к моделям, и для каждого пути определяется формат файла (по расширению). Каждая модель загружается через соответствующий импортёр (`GLBImporter` или `STLImporter`) и сохраняется во временном списке. Затем для каждого элемента из массива игровых объектов извлекаются индекс модели, тег и параметры трансформации. В зависимости от типа модели создаётся экземпляр соответствующего класса игрового объекта (`GLBGameObject` или `STLGameObject`). Созданному объекту присваивается тег, а затем через менеджер игровых объектов он добавляется на сцену. После добавления объекту устанавливаются параметры трансформации: позиция, поворот и масштаб, извлечённые из JSON.

Управление ресурсами при загрузке. Важной особенностью реализации является то, что модели загружаются один раз для всей сцены и затем используются многократно. Это позволяет экономить видеопамять и избегать дублирования геометрических данных. При удалении сцены или завершении приложения все загруженные модели и связанные с ними ресурсы OpenGL освобождаются через механизм очистки.

Обработка ошибок. При десериализации сцены предусмотрена обработка ошибочных ситуаций: если файл модели по указанному пути не существует, в консоль выводится сообщение об ошибке и программа завершается. При отсутствии файла сцены в целом процесс загрузки не выполняется, и сцена инициализируется в пустом состоянии, что позволяет избежать аварийного завершения приложения.

3.9. Подсистема обработки ввода

3.9.1. Работа хуков

Под хуками в контексте движка понимаются методы-обработчики, которые вызываются в ответ на определённые события жизненного цикла или действия пользователя. Все хуки определены как виртуальные методы в базовом классе сцены, что позволяет разработчику переопределять их в конкретных сценах для реализации специфической логики.

Источники вызовов. Хуки вызываются из двух основных мест: из класса `GameApplication` и из менеджера сцен. `GameApplication` является точкой входа приложения и получает события от операционной системы через колбэки `GLFW`. При возникновении события (нажатие клавиши, движение мыши, обновление кадра) `GameApplication` вызывает соответствующий метод у текущей сцены. Менеджер сцен, в свою очередь, вызывает хуки жизненного цикла (старт, выход, пауза, возобновление) в моменты переключения сцен или изменения состояния приложения.

Жизненный цикл сцены. При первой загрузке сцены менеджер сцен вызывает хук `OnStart`. Этот метод предназначен для однократной инициализации логики сцены, создания начальных объектов и подписки на события. При выгрузке сцены вызывается хук `OnExit`, который даёт возможность выполнить завершающие действия: сохранить состояние, освободить временные ресурсы и т.д. При приостановке сцены (например, при открытии меню паузы) вызывается хук `OnPause`, а при возобновлении — `OnResume`. Это позволяет корректно управлять игровым процессом без потери данных.

Игровой цикл. Основной игровой цикл реализован в классе `GameApplication`. На каждом кадре последовательно вызываются хуки `Update` и `FixedUpdate`. Хук `Update` вызывается с параметром `deltaTime`, представляющим время, прошедшее с предыдущего кадра. Этот хук предназначен для обработки логики, зависящей от частоты кадров: движения объектов, анимации, проверки условий. Хук `FixedUpdate` вызывается с фиксированным временным интервалом, что обеспечивает детерминиро-

ванное поведение физических расчётов независимо от производительности оборудования.

Обработка ввода. При получении события от операционной системы `GameApplication` вызывает соответствующий хук у текущей сцены. Реализованы следующие хуки ввода: `OnKeyDown` и `OnKeyUp` для событий клавиатуры; `OnMouseDown`, `OnMouseUp`, `OnMouseMove` и `OnMouseWheel` для событий мыши. Каждый хук получает аргументы события, содержащие подробную информацию: код нажатой клавиши, модификаторы, координаты курсора, величину прокрутки колеса.

Распространение событий на игровые объекты. Ключевой особенностью реализации является то, что все перечисленные хуки в базовом классе сцены не содержат собственной логики, а лишь выполняют обход всех игровых объектов, зарегистрированных на сцене, и вызывают соответствующий метод у каждого объекта. Это позволяет любому игровому объекту переопределить нужные хуки и реагировать на события без необходимости регистрации в диспетчере событий или подписки на делегаты. Такой подход упрощает разработку игровой логики: разработчику достаточно создать класс, унаследованный от `GameObject`, и переопределить нужные виртуальные методы.

Порядок вызовов. Для каждого события гарантируется определённый порядок вызовов. В случае хуков жизненного цикла и игрового цикла все игровые объекты обрабатываются в порядке их добавления на сцену. Для событий ввода порядок не регламентируется, но гарантируется, что все объекты получат событие. Это позволяет реализовывать простые сценарии взаимодействия без необходимости управления очередью обработчиков.

Особенности реализации в редакторе. При работе в режиме редактирования хуки не вызываются, чтобы разработчик мог спокойно редактировать сцену. При переключении в режим игры работа игрового цикла возобновляется, и все хуки начинают вызываться в штатном режиме.

3.9.2. Рейкастинг

Рейкастинг представляет собой механизм определения объекта в трёхмерной сцене, на который указывает курсор мыши. В разработанном движке этот механизм используется для реализации интерактивного взаимодействия пользователя с объектами сцены, например, для выбора объекта кликом мыши.

Принцип работы. Рейкастинг основан на построении луча из камеры через позицию курсора на экране и проверке пересечения этого луча с ограничивающими объёмами объектов. Луч задаётся начальной точкой (позиция камеры) и направлением, которое вычисляется на основе координат курсора, размера окна и параметров проекции камеры. Направление луча определяется таким образом, чтобы он проходил через точку на ближней плоскости отсечения, соответствующую положению курсора, и уходил вглубь сцены.

Построение луча. Метод `GetPickRay` класса `Camera` принимает координаты курсора в оконной системе координат, ширину и высоту окна, а также матрицы проекции и вида. Вычисление выполняется в несколько этапов: преобразование оконных координат в нормализованные координаты устройства, затем в координаты пространства камеры и, наконец, в мировые координаты. Полученный луч используется для проверки пересечения с объектами сцены. Все математические операции выполняются с использованием матричных преобразований, что обеспечивает корректную работу при любых настройках камеры.

Проверка пересечения. Для каждого игрового объекта на сцене вызывается метод `RayIntersectsAABB`, который проверяет, пересекает ли луч ограничивающий прямоугольник объекта в мировых координатах. Использование ограничивающего прямоугольника, а не точной геометрии модели, является компромиссом между точностью и производительностью. Вычисление точного пересечения с каждым полигоном модели потребовало бы значительно больше вычислительных ресурсов и могло бы привести к снижению частоты кадров при большом количестве объектов.

Выбор ближайшего объекта. Поскольку луч может пересекать несколько объектов одновременно (например, когда один объект находится за другим), необходимо определить, какой из них расположен ближе к камере. Алгоритм перебирает все объекты сцены, для каждого вычисляет расстояние от начала луча до точки пересечения и сохраняет объект с минимальным положительным расстоянием. Расстояние считается положительным только в том случае, если точка пересечения находится перед камерой (а не позади неё). В результате возвращается объект, ближайший к камере среди всех пересечённых.

Обработка результатов. Метод `PickObject` возвращает объект типа `DrawableObject`, если пересечение найдено, или `null` в противном случае. Полученный объект может быть использован для дальнейшей обработки: подсветки, удаления, изменения свойств или выполнения игровой логики. В демонстрационном приложении результат рейкастинга используется для выбора объекта при клике мыши.

Производительность. В текущей реализации рейкастинг выполняется с линейной сложностью относительно количества объектов на сцене, так как перебираются все объекты. При сценах с небольшим количеством объектов (до нескольких сотен) этого достаточно для поддержания 60 кадров в секунду. Для сцен с тысячами объектов требуется дополнительная оптимизация, например, пространственное разбиение сцены, но в рамках дипломной работы такая оптимизация не реализована.

3.10. Система импорта трёхмерных моделей

Система импорта трёхмерных моделей отвечает за загрузку файлов внешних форматов, их парсинг и преобразование во внутреннее представление движка, пригодное для использования в рендеринге.

3.10.1. Поддерживаемые форматы (STL, GLB)

В разработанном игровом движке реализована поддержка двух форматов трёхмерных моделей: STL и GLB.

Формат STL (Stereolithography) является широко распространённым форматом для хранения трёхмерных моделей, особенно в области 3D-печати и инженерных расчётов. STL представляет геометрию объекта в виде набора треугольников, каждый из которых описывается тремя вершинами и вектором нормали. Формат существует в двух вариантах: текстовом (ASCII) и бинарном (Binary). Текстовый STL легко читается человеком и отлаживается, но занимает больше места на диске. Бинарный STL более компактен и быстрее загружается, поскольку не требует парсинга текстовых чисел. STL-файлы не поддерживают иерархию узлов, материалы, текстурные координаты и анимацию, что ограничивает их применение в сложных игровых проектах. Однако простота формата позволяет быстро импортировать модели, созданные в САПР-системах, что делает его удобным для прототипирования и демонстрационных целей.

Формат GLB является бинарной версией формата glTF (GL Transmission Format), разработанного консорциумом Khronos Group как современный стандарт для передачи трёхмерных сцен в веб-приложениях и игровых движках. В отличие от STL, GLB является полнофункциональным форматом, поддерживающим: иерархическую структуру узлов с матрицами трансформации; несколько сеток в одном файле; материалы с настраиваемыми параметрами (цвет, металличность, шероховатость); текстурные координаты и загрузку текстур; анимацию трансформаций; скелетную анимацию (включая кости и веса вершин). GLB представляет собой один бинарный файл, в котором объединены JSON-описание сцены и бинарные данные геометрии, что упрощает распространение моделей, так как все компоненты хранятся в одном файле, а не в наборе файлов. Этот формат выбран в качестве основного для сложных моделей, требующих поддержки материалов и иерархии. Устройство файла показано на рисунке 3.4

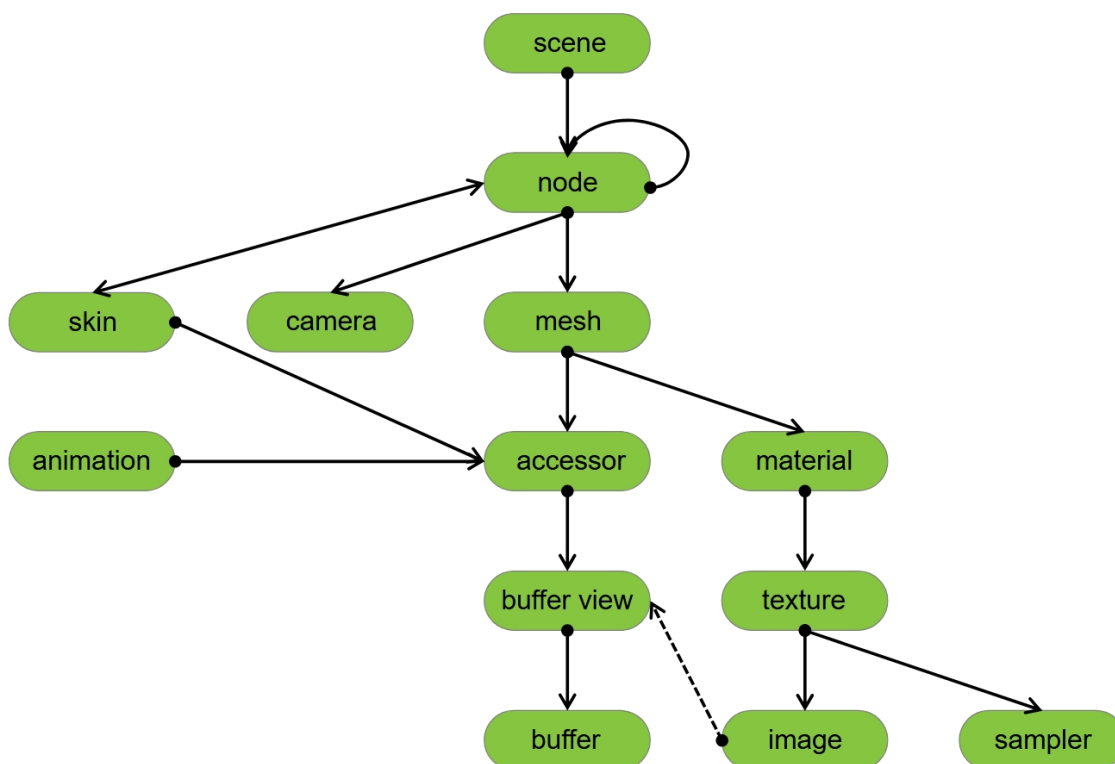


Рисунок 3.4 – Структура glb файла

Выбор форматов. Поддержка STL и GLB обусловлена их взаимодополняющими характеристиками. STL используется для быстрого импорта простых моделей и прототипирования, а также для совместимости с системами, экспортирующими только этот формат. GLB применяется для полноценных игровых моделей с материалами и анимацией. Такая комбинация покрывает широкий спектр задач от инженерных демонстраций до полноценных игр.

3.10.2. Парсинг данных из STL-файлов

Импорт STL-файлов в движке реализован с использованием бинарного чтения, что обеспечивает высокую производительность даже при загрузке моделей с большим количеством треугольников. Процесс парсинга начинается с проверки существования файла по указанному пути; при отсутствии файла в консоль выводится сообщение об ошибке, а загрузка прерывается.

Определение типа STL-файла выполняется путём чтения первых пяти байт. Если файл начинается с последовательности "solid", он идентифици-

руется как текстовый STL. В текущей версии движка текстовый формат не поддерживается, так как его обработка требует значительно больше вычислительных ресурсов и времени, а область применения таких моделей в игровой разработке ограничена. При обнаружении текстового формата в консоль выводится сообщение о неподдерживаемом формате, и загрузка прекращается. Все поддерживаемые файлы обрабатываются как бинарные STL.

После определения формата указатель чтения возвращается в начало файла, и считывается 80-байтовый заголовок, который обычно содержит произвольную текстовую информацию о модели и может быть проигнорирован. Затем извлекается 4-байтовое целое без знака, указывающее количество треугольников в модели. Перед загрузкой выполняется проверка: если количество треугольников превышает заданный лимит (один миллион), загрузка прерывается с выводом сообщения об ошибке, что предотвращает чрезмерное потребление памяти.

Для хранения геометрических данных создаётся структура, содержащая массивы вершин и нормалей. Вершины и нормали хранятся в виде плоских массивов байтов, что позволяет эффективно копировать данные из файла и впоследствии передавать их в OpenGL. Размер массивов вычисляется заранее на основе количества треугольников: каждый треугольник содержит три вершины, каждая вершина описывается тремя координатами с плавающей точкой, что даёт девять чисел с плавающей точкой на треугольник.

В цикле по всем треугольникам последовательно считываются данные: сначала нормаль треугольника (три числа с плавающей точкой), затем координаты трёх вершин (девять чисел с плавающей точкой). Нормаль записывается в массив нормалей трижды — по одному разу для каждой вершины треугольника, что упрощает последующее использование в шейдерах, где нормаль требуется для каждой вершины. Координаты вершин копируются в массив вершин.

После чтения вершин и нормалей считывается 2-байтовый атрибут, который в стандарте STL зарезервирован для пользовательских данных. В некоторых STL-файлах в этом поле кодируется цвет треугольника в формате

RGB555 (5 бит на каждый цветовой канал). Движок поддерживает извлечение цветов из этого поля: выделяется 5 бит на красный, зелёный и синий каналы, каждый из которых преобразуется в значение с плавающей точкой в диапазоне от 0 до 1. Альфа-канал устанавливается в единицу (полностью непрозрачный). Если в файле обнаружены цветовые данные, для модели активируется режим вершинных цветов, который может быть использован в шейдере вместо или в дополнение к текстурам. Если цветовые данные отсутствуют, соответствующая опция рендеринга отключается.

После успешного чтения всех треугольников выполняется проверка, что указатель чтения достиг конца файла. Если после обработки всех треугольников в файле остались непрочитанные данные, файл считается повреждённым, и загрузка прерывается с сообщением об ошибке.

На завершающем этапе парсинга вычисляется ограничивающий прямоугольник модели. Для этого обходятся все загруженные вершины, и для каждой определяются минимальные и максимальные значения по каждой оси. Полученный прямоугольник будет использоваться для рейкастинга и отсечения невидимых объектов. На основе собранных данных создаётся объект модели, который сохраняет путь к исходному файлу для возможной перезагрузки и настройки рендеринга.

Все операции с файлом выполняются с использованием буферизованного чтения, что минимизирует количество обращений к диску. При возникновении любой ошибки в процессе чтения (неверный формат, преждевременный конец файла, повреждённые данные) загрузка прерывается, и в консоль выводится диагностическое сообщение, позволяющее определить причину сбоя.

3.10.3. Парсинг данных и иерархии узлов из GLB-файлов

Импорт GLB-файлов в разработанном движке реализован в соответствии со спецификацией glTF 2.0. GLB-файл представляет собой бинарный контейнер, объединяющий JSON-описание сцены и бинарные данные геометрии в одном файле. Процесс парсинга начинается с проверки существо-

вания файла и его размера: файлы размером более одного гигабайта не загружаются для предотвращения чрезмерного потребления памяти.

Первым этапом является чтение заголовка GLB-файла, который содержит магическое число (0x46546C67, ASCII-строка "glTF"), версию формата и размер файла. Движок поддерживает только версию 2.x, так как более старые версии формата не обеспечивают необходимой функциональности. При обнаружении неподдерживаемой версии загрузка прерывается с выводом сообщения об ошибке.

После заголовка последовательно считываются блоки данных (chunks). Первый блок является JSON-блоком, содержащим описание всей сцены в текстовом формате. Этот блок парсится в JSON-документ, который в дальнейшем используется для извлечения структуры сцены. Второй блок является бинарным блоком и содержит буферы с вершинными данными — позициями, нормальями, текстурными координатами и индексными данными.

Из JSON-дескриптора извлекаются массивы буферов, буферов представления (buffer views) и аксессоров (accessors). Буферы представляют собой сырые бинарные данные. Буферы представления определяют, какой участок буфера и с каким шагом между элементами должен использоваться. Аксессоры описывают, как интерпретировать данные из буфера представления: тип компонента (целое число со знаком/без знака, число с плавающей точкой), количество компонентов в элементе (скаляр, вектор из двух, трёх или четырёх компонентов), нормализованы ли значения, а также общее количество элементов. Аксессоры также могут содержать минимальные и максимальные значения координат, что позволяет вычислить ограничивающий прямоугольник модели без обхода всех вершин.

Особенностью реализации является поддержка разреженных аксессоров (sparse accessors), которые используются для эффективного хранения данных, где большинство значений являются дефолтными. При обнаружении разреженного аксессора движок копирует базовые данные, а затем заменяет элементы по указанным индексам значениями из разреженного хранилища.

Это позволяет корректно загружать модели, оптимизированные с использованием данной техники.

Из JSON-дескриптора также извлекаются описания мешей, каждый из которых может содержать несколько примитивов. Примитив определяет тип геометрии (обычно треугольники), аксессор индексов и аксессоры атрибутов вершины: позиции (обязательный), нормали, текстурные координаты (до пяти наборов). Для каждого примитива может быть указан материал, определяющий внешний вид поверхности. На основе этих данных для каждого примитива создаётся структура, содержащая массивы вершин, индексов, нормалей, текстурных координат, а также ограничивающий прямоугольник.

Иерархия узлов (nodes) извлекается из JSON-дескриптора и реконструируется в граф сцены. Каждый узел может содержать дочерние узлы, матрицу трансформации или отдельные компоненты трансформации (позицию, поворот в виде кватерниона, масштаб). Для каждого узла вычисляется локальная матрица трансформации, а затем рекурсивно вычисляются глобальные матрицы с учётом трансформаций всех родительских узлов. Узлы, содержащие меши, преобразуются в игровые объекты с соответствующими компонентами трансформации и визуального отображения. Узлы, являющиеся костями скелетной анимации, идентифицируются и могут быть использованы для анимации (хотя сама анимация в текущей версии не поддерживается).

В таблице 3.5 показано, то, какие возможности формата GLB поддерживает движок:

Таблица 3.5 – Поддерживаемые возможности формата GLB в разработанном импортере

Возможность	Описание поддержки
Версия формата	Поддерживается только версия 2.x. При обнаружении других версий загрузка прерывается с выводом сообщения об ошибке.

Продолжение таблицы 3.5

Возможность	Описание поддержки
Буферы (buffers)	Полная поддержка: чтение из бинарного блока GLB и из внешних URI в кодировке base64.
Буферы представления (buffer views)	Полная поддержка: извлечение данных с указанным смещением, длиной и шагом между элементами.
Аксессоры (accessors)	Полная поддержка: все типы компонентов (целые числа со знаком/без знака, числа с плавающей точкой), нормализация, все типы данных (скаляр, векторы).
Разреженные аксессоры (sparse accessors)	Поддерживаются. Выполняется замена элементов по указанным индексам.
Примитивы (primitives)	Поддерживаются все возможные типы.
Позиции вершин (POSITION)	Обязательный атрибут. Используется для построения геометрии и ограничивающего прямоугольника.
Нормали (NORMAL)	Поддерживаются.
Текстурные координаты (TEXCOORD_0 — TEXCOORD_4)	Поддерживаются до пяти наборов.
Материалы (materials)	Поддерживается модель PBR (Metallic-Roughness).
Базовый цвет (baseColorFactor)	Поддерживается. Задаётся в виде RGBA-вектора.
Карта базового цвета (baseColorTexture)	Поддерживается.
Металличность (metallicFactor)	Поддерживается. Значение от 0 до 1.

Продолжение таблицы 3.5

Возможность	Описание поддержки
Шероховатость (roughnessFactor)	Поддерживается. Значение от 0 до 1.
Карта металличности и шероховатости (metallicRoughnessTexture)	Поддерживается.
Карта нормалей (normalTexture)	Поддерживается с учётом масштаба (scale).
Карта окклюзии (occlusionTexture)	Поддерживается с учётом силы эффекта (strength).
Карта эмиссии (emissiveTexture)	Поддерживается.
Фактор эмиссии (emissiveFactor)	Поддерживается. Задаётся в виде RGB-вектора.
Семплеры (samplers)	Поддерживаются параметры фильтрации (магнитная и минимальная) и режимы наложения по осям S и T (wrapS, wrapT).
Изображения (images)	Поддерживаются изображения, встроенные в буферы через base64.
Текстуры (textures)	Полная поддержка. Связывают изображение с семплером.
Узлы (nodes)	Полная поддержка иерархии, матриц трансформации и отдельных компонентов (позиция, поворот в виде кватерниона, масштаб).
Сцены (scenes)	Поддерживаются все сцены в файле. Каждая сцена импортируется как отдельная модель.
Скелетная анимация (skins, joints)	Идентифицируются узлы-кости, но сама анимация не воспроизводится.

Продолжение таблицы 3.5

Возможность	Описание поддержки
Ограничение на размер файла	Максимальный размер файла — 1 гигабайт. При превышении загрузка прерывается.

После завершения парсинга всех компонентов для каждой сцены рекурсивно обходится граф узлов, и для каждого узла, содержащего меш, создаётся модель, которая объединяет все меши узла и всех его потомков. Каждая полученная модель сохраняет путь к исходному файлу и свой индекс в сцене, что позволяет в дальнейшем ссылаться на неё при сериализации сцены. Весь процесс импорта полностью управляем: при возникновении любой ошибки (неверный формат, отсутствие обязательных полей, повреждённые данные) загрузка прерывается, а в консоль выводится диагностическое сообщение.

3.10.4. Выводы по разделу

В рамках технического проекта определено внутреннее устройство разработанного игрового движка на языке C# с использованием графического API OpenGL. Обоснован выбор C#, OpenGL, Windows Forms, SixLabors.ImageSharp и библиотек для импорта STL и GLB в качестве технологической основы проекта. Описана архитектура движка, включающая графическую подсистему, систему управления игровыми объектами, менеджер сцен, систему камер и освещения, а также редактор сцен. Подробно рассмотрено устройство системы трансформации с использованием матриц преобразований и кватернионов, включая формулы вычисления матриц модели, вида и проекции. Описана графическая подсистема: управление вершинными и индексными буферами, компиляция шейдеров с динамической генерацией макросов на основе характеристик материала и модели освещения, а также конвейер рендеринга. Отдельно рассмотрена система импорта трёхмерных моделей из форматов STL и GLB с поддержкой иерархии узлов, материалов, текстур и разреженных аксессоров.

В качестве практического подтверждения проектных решений рассмотрено демонстрационное приложение — трёхмерная игра, в которой пользователь нажимает на объекты, после чего они исчезают и возрождаются в случайных позициях. Приложение демонстрирует совместную работу обработки ввода, рейкастинга для выбора объектов, системы трансформации и рендеринга, что подтверждает работоспособность разработанного движка.

Список литературы

1. Ферроне, Х. Изучаем C# через разработку игр на Unity. 5-е издание / Х. Ферроне. – Санкт-Петербург : Питер, 2026. – 400 с. – ISBN 978-5-4461-2932-4. – Текст : непосредственный.
2. Горелов, С. В. Современные технологии программирования: разработка Windows-приложений на языке C# / С. В. Горелов ; под ред. П. Б. Лукьянова. – Москва : Прометей, 2019. – 362 с. – Текст : непосредственный.
3. Патрашов, А. Математическое руководство по созданию компьютерных игр. Справочник / А. Патрашов. – Москва : Издательские решения, 2016. – 365 с. – ISBN 978-5-4483-2283-9. – Текст : непосредственный.
4. Вольф, Д. OpenGL 4. Язык шейдеров. Книга рецептов / Д. Вольф ; пер. с англ. А. Н. Киселёва. – 2-е изд., эл. – Москва : ДМК Пресс, 2023. – 370 с. – ISBN 978-5-89818-587-9. – Текст : непосредственный.
5. Снетков, В. М. Разработка приложений на C# в среде Visual Studio 2005 / В. М. Снетков. – Москва : Национальный Открытый Университет ИНТУИТ, 2024. – 956 с. – Текст : электронный.
6. Шрайнер, Д. OpenGL. Официальное руководство программиста. 9-е издание / Д. Шрайнер, Г. Селле. – Москва : ДМК Пресс, 2022. – 1100 с. – ISBN 978-5-97060-987-5. – Текст : непосредственный.
7. Ленгель, Э. Математика для трёхмерной компьютерной графики и разработки игр / Э. Ленгель. – Москва : Вильямс, 2019. – 524 с. – ISBN 978-5-9909446-2-6. – Текст : непосредственный.
8. Шрайнер, Д. OpenGL. Язык шейдеров GLSL. 3-е издание / Д. Шрайнер, Г. Селле. – Москва : ДМК Пресс, 2021. – 480 с. – ISBN 978-5-97060-945-5. – Текст : непосредственный.
9. Грегори, Дж. Архитектура игрового движка. Введение в разработку игрового программного обеспечения / Дж. Грегори. – Москва : ДМК Пресс, 2020. – 1200 с. – ISBN 978-5-97060-926-4. – Текст : непосредственный.

10. Данн, Ф. Трёхмерная математика для компьютерной графики и разработки игр / Ф. Данн, И. Парберри. – Москва : ДМК Пресс, 2019. – 560 с. – ISBN 978-5-97060-754-3. – Текст : непосредственный.
11. Тюкачев, Н. А. C#. Программирование 2D и 3D векторной графики / Н. А. Тюкачев, В. Г. Хлебостроев. – 5-е изд., стер. – Санкт-Петербург : Лань, 2024. – 320 с. – ISBN 978-5-507-47565-0. – Текст : непосредственный.
12. Шикин, Е. В. Компьютерная графика. Полигональные модели / Е. В. Шикин, А. В. Боресков. – Москва : ДИАЛОГ-МИФИ, 2000. – 461 с. – ISBN 5-86404-139-4. – Текст : непосредственный.
13. Luchshev, P. A. Computer graphics: tutorial / P. A. Luchshev, N. G. Gulub. – Kharkiv : KhAI, 2024. – 80 p. – URL: <http://dspace.library.khai.edu/xmlui/handle/123456789/8641> (дата обращения: 17.05.2026). – Текст : электронный.
14. The Open Toolkit Library (OpenTK). – URL: <https://github.com/opentk/opentk> (дата обращения: 19.04.2026). – Текст : электронный.
15. GLB File Format Specification. – URL: <https://docs.fileformat.com/ru/3d/glb/> (дата обращения: 01.05.2026). – Текст : электронный.
16. STL Format Description. – URL: <https://industry3d.ru/formaty-dannykh/> (дата обращения: 11.05.2026). – Текст : электронный.
17. OpenGL – The Industry’s Foundation for High Performance Graphics / The Khronos Group. – URL: <https://www.khronos.org/opengl/> (дата обращения: 01.05.2026). – Текст : электронный.